

HiDiTingV100 IDE 工具

使用指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档描述了 IDE 工具的安装及使用，指导用户快速完成代码的编译烧录等工作。IDE 工具使用的是 DevEco Device Tool，该工具主要用于代码的编辑、编译、镜像烧写。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 硬件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 工具简介.....	1
2 开发环境搭建	3
2.1 概述.....	3
2.2 DevEco Device Tool 软件安装	3
3 工程管理.....	13
4 编译运行.....	17
4.1 编译 BrandyK100 工程.....	17
4.2 自定义编译命令方法介绍	19
5 软件包烧录.....	22
5.1 烧录工具配置	22
5.2 烧录工程配置	24
5.3 执行工程烧录	26
6 分 bin 烧录.....	28
7 栈分析和镜像分析	33
7.1 路径配置.....	33
7.2 栈分析	37
7.3 镜像分析.....	39
8 添加文件到编译框架.....	43
8.1 生成 CMake 文件模板	43
8.2 插入 CMake 文件模板	45

8.3 插入 CMake 日志	47
8.4 自动添加文件到编译框架	48
9 工程调试.....	54
9.1 调试工具配置	54
9.2 调试配置选项	57
9.3 启动调试.....	58
9.4 常用调试功能	59
9.5 调试实例介绍	63
10 常见错误.....	68
10.1 SDK 根目录路径过长	68
10.2 路径失效.....	69
10.3 调试时出现读取内存地址失败	69
10.4 IDE 兼容性问题	70
10.5 替换编译工具链后编译报错	73
10.6 调试异常打印【warning】Couldn't determine a path for the index cache directory	74
10.7 自定义编译命令后调试出现问题	75

插图目录

图 1-1 功能分区图..... 2

图 2-1 扩展功能按钮 8

图 2-2 Visual Studio Code 简体中文插件安装失败示例..... 8

图 2-3 Visual Studio Code 重启后界面 12

图 6-1 空板烧录失败 30

1 工具简介

DevEco Device Tool 是 OpenHarmony 面向智能设备开发者提供的一站式集成开发环境，支持 OpenHarmony 的组件按需定制，支持代码编辑、编译、烧录和调试等功能，支持 C/C++ 语言，以插件的形式部署在 Visual Studio Code 上。

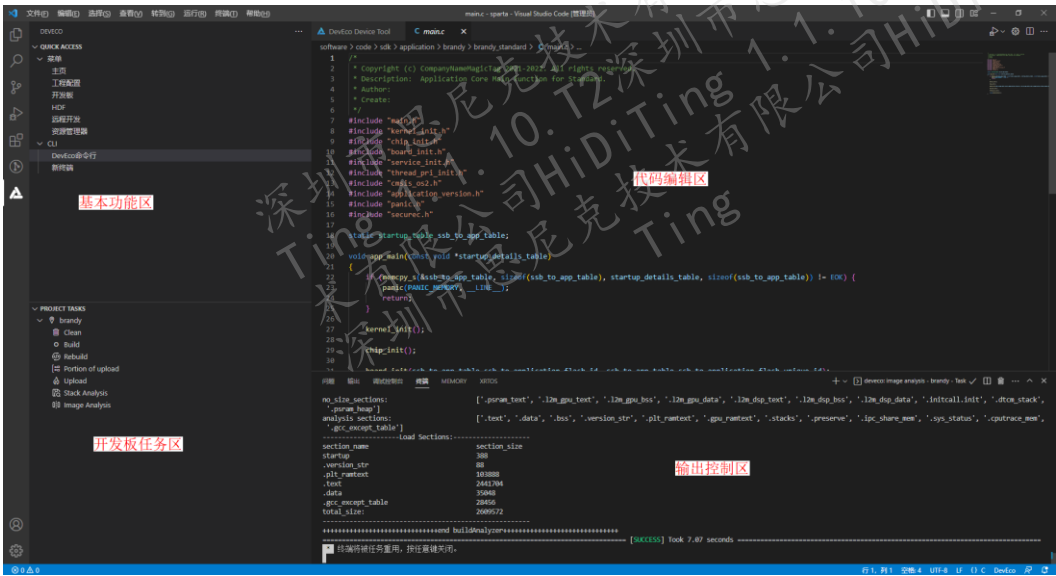
DevEco Device Tool 支持 Windows 开发环境，在 Windows 上主要进行代码开发、工程编译、代码调试、镜像烧录等操作。其具有以下特点：

- 支持代码查找、代码高亮、代码自动补齐、代码输入提示、代码检查等，开发者可以轻松、高效编码。
- 支持自动检测各芯片/开发板依赖的工具链是否完备，并提供一键下载和安装缺失工具链。
- 支持源码级调试能力，提供查看内存、变量、调用栈、寄存器、反汇编等调试信息。

DevEco Device Tool 工具主要分为如图 1-1 所示 4 个功能区域。

- 基本功能区：DevEco Device Tool 菜单栏，提供基本的工程创建、源码导入、工程配置等功能。
- 开发板任务区：提供开发板相关操作任务，如源码的编译、镜像的烧录等。
- 代码编辑区：提供代码的查看、编写和调试等开发功能。
- 输出控制区：提供日志打印、调试指令输入等。

图1-1 功能分区图



2 开发环境搭建

2.1 概述

2.2 DevEco Device Tool 软件安装

2.1 概述

BrandyK100 项目代码编辑、编译、烧录工作全部在 Windows 系统下完成，本文档主要介绍 Windows 上的开发环境搭建与使用 DevEco Device Tool 开发 BrandyK100 代码。

Windows 系统要求：

- Windows10 64 位系统及以上

环境搭建要求：

- DevEco Device Tool 安装包：deveco-device-tool-all-in-one-local.exe
- DevTools_CFBB 工具包

2.2 DevEco Device Tool 软件安装

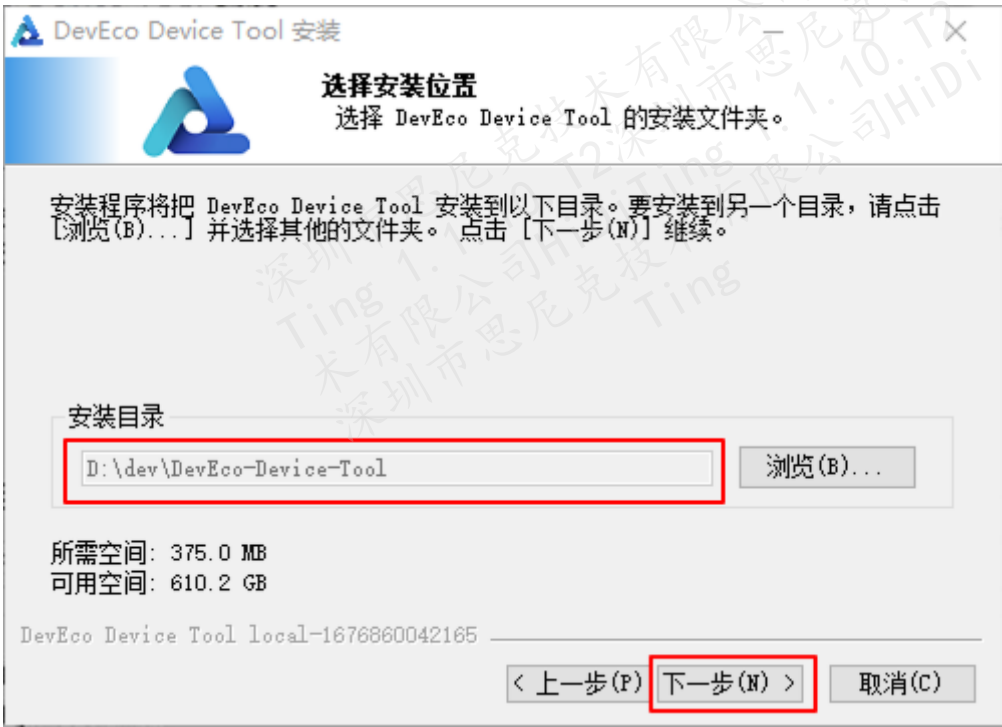
步骤 1 双击 deveco-device-tool 安装包程序，点击“下一步”进行安装。



步骤 2 在这一步用户可以点击“用户协议”和“隐私声明”查看其具体内容。如果没有异议，可选择“我接受许可证协议中的条款”，并点击“下一步”进行安装。



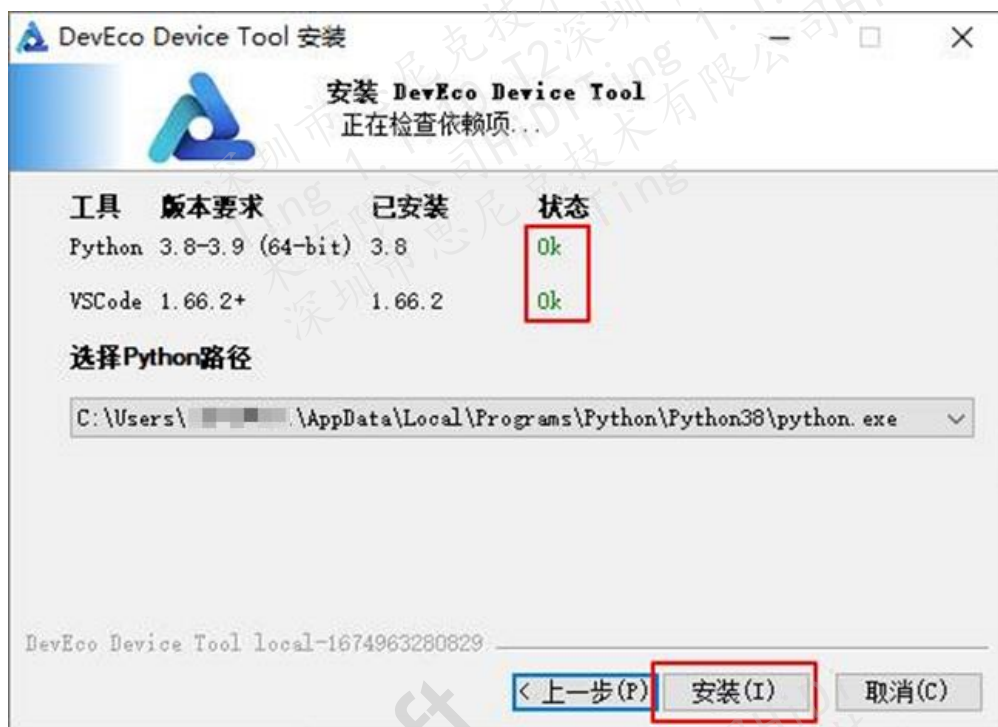
步骤 3 设置 DevEco Device Tool 的安装路径。点击“浏览”按钮可以自主选择安装路径，但是请注意安装路径不能包含中文字符和特殊字符（例如：#,?~^0[]{}\$），如果安装路径包含中文字符和特殊字符，会导致安装失败。点击“下一步”。



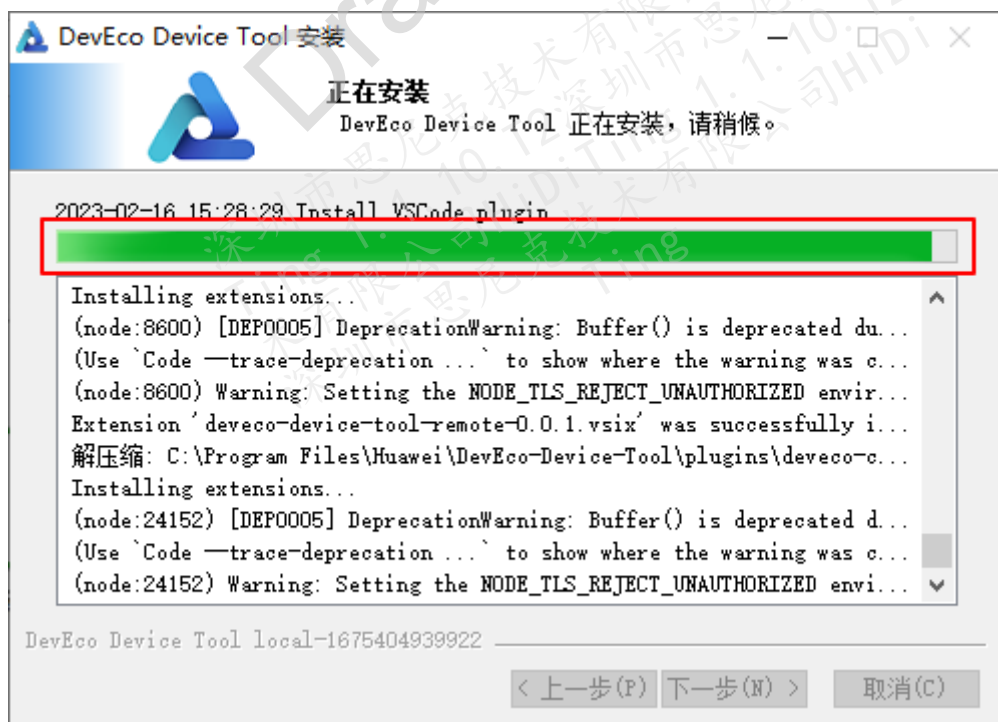
步骤 4 DevEco Device Tool 工具依赖 python 和 Visual Studio Code 软件。此步骤主要是检测这两个软件是否已安装以及安装版本是否符合工具需求。如果没有安装 python 或 Visual Studio Code 软件，点击“安装”按钮，软件会自动进行下载和安装（点击“自定义安装”按钮，软件会进行自动下载，需要用户进行手动安装）。



1. Python 和 Visual Studio Code 软件安装好之后，状态会变成“ok”，然后点击“安装”按钮。



2. 等待程序的安装进度条执行到 100%，会自动切换安装画面到下一步。



3. 点击“完成”，关闭 DevEco Device Tool 安装向导。



步骤 5 以管理员身份运行 Visual Studio Code，DevEco Device Tool 工具界面自动打开。至此 DevEco Device Tool Windows 开发环境安装完成。后续用户可以在 Visual Studio Code 中输入快捷键“Ctrl+Shift+X”或点击扩展功能按钮（具体位置如图 2-1 所示）打开扩展功能，在扩展中搜索“Chinese (Simplified) (简体中文) Language Pack for Visual Studio Code”插件并安装，安装完成后重启 Visual Studio Code，即可将 VSCode 设置为中文模式。如果出现图片中安装失败的情况如图 2-2 所示，请根据如下操作安装中文插件。

图2-1 扩展功能按钮

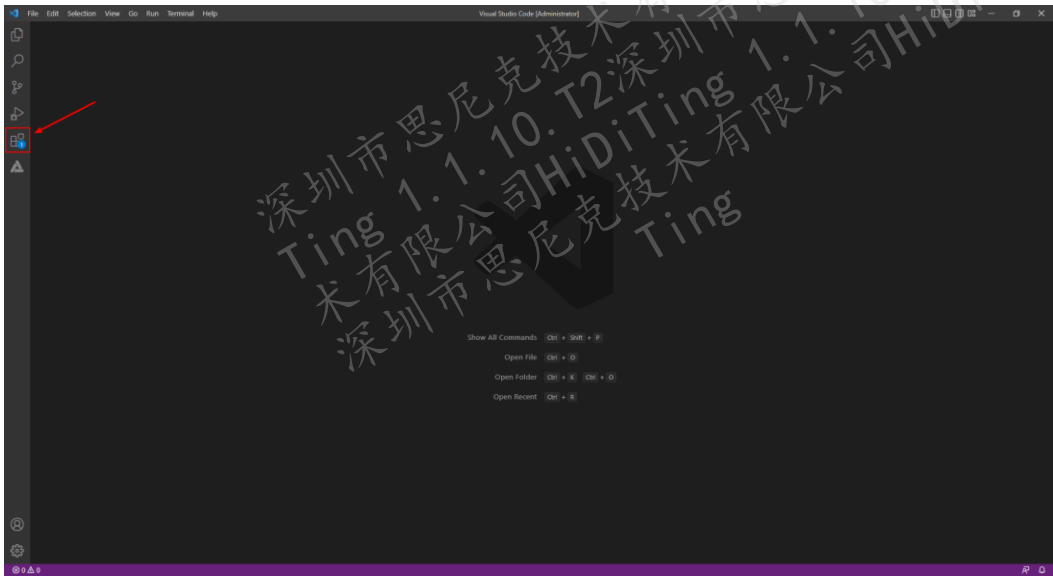
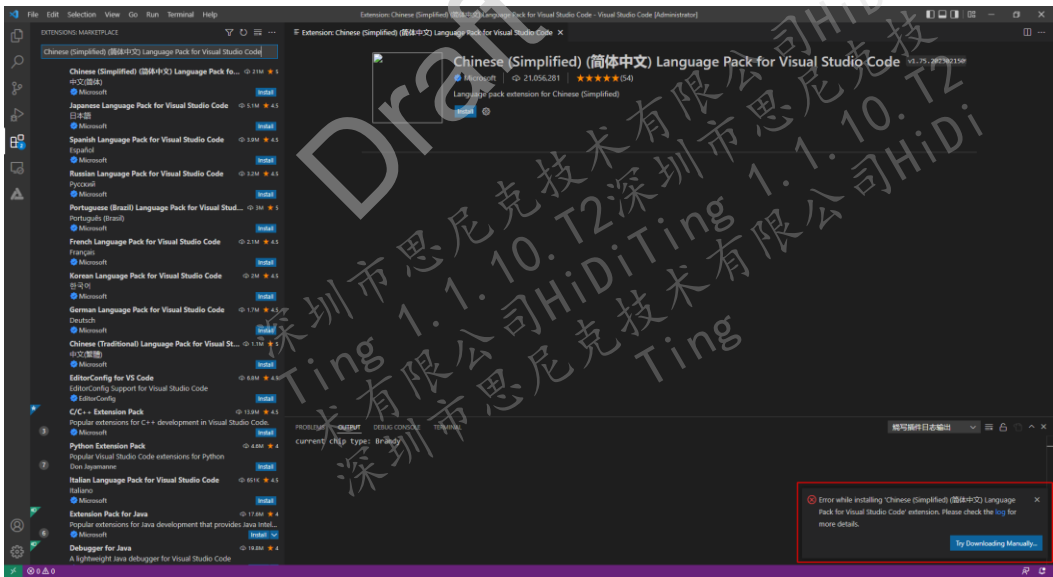
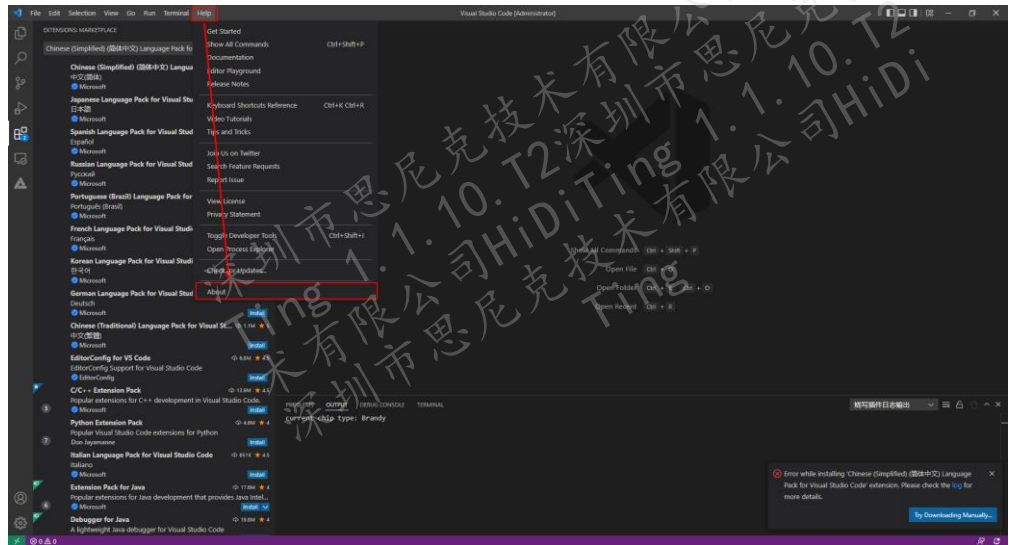


图2-2 Visual Studio Code 简体中文插件安装失败示例

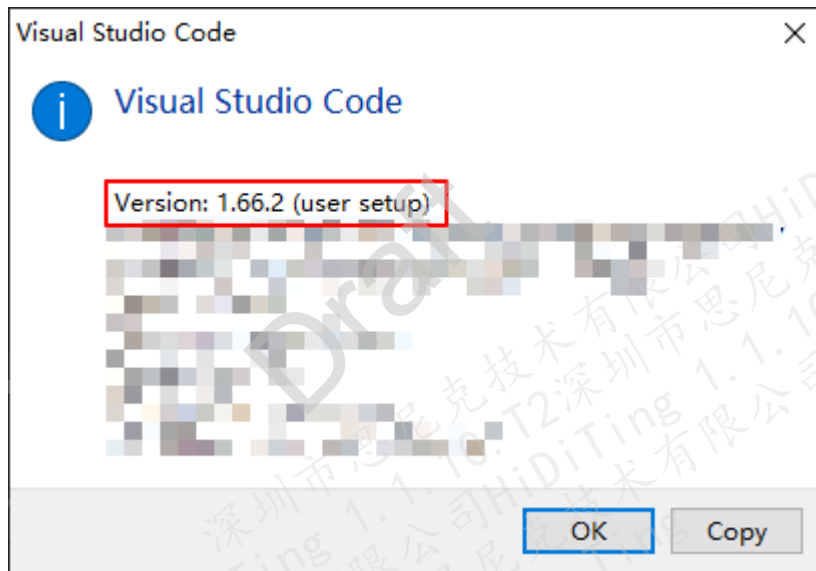


安装中文插件具体操作方法：

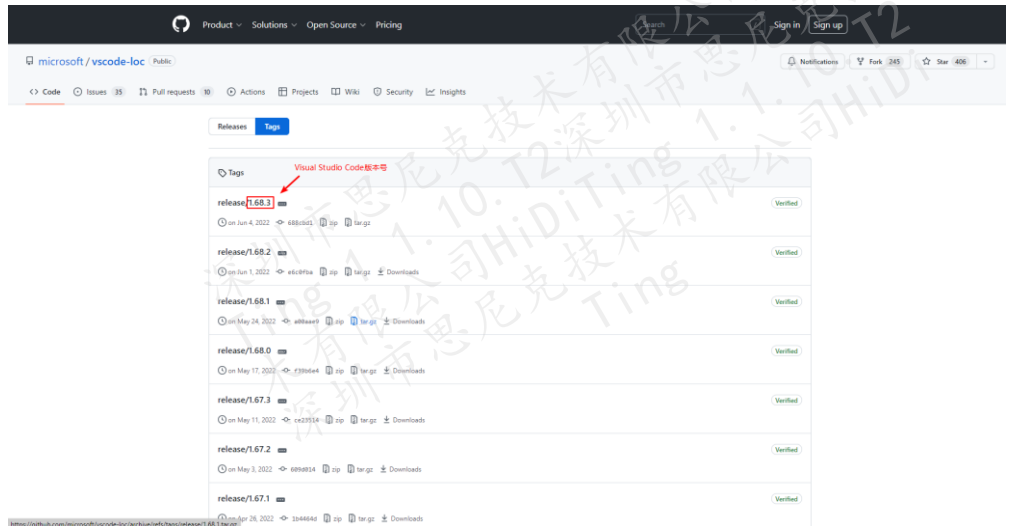
1. 点击 Visual Studio Code 中的 Help→About。



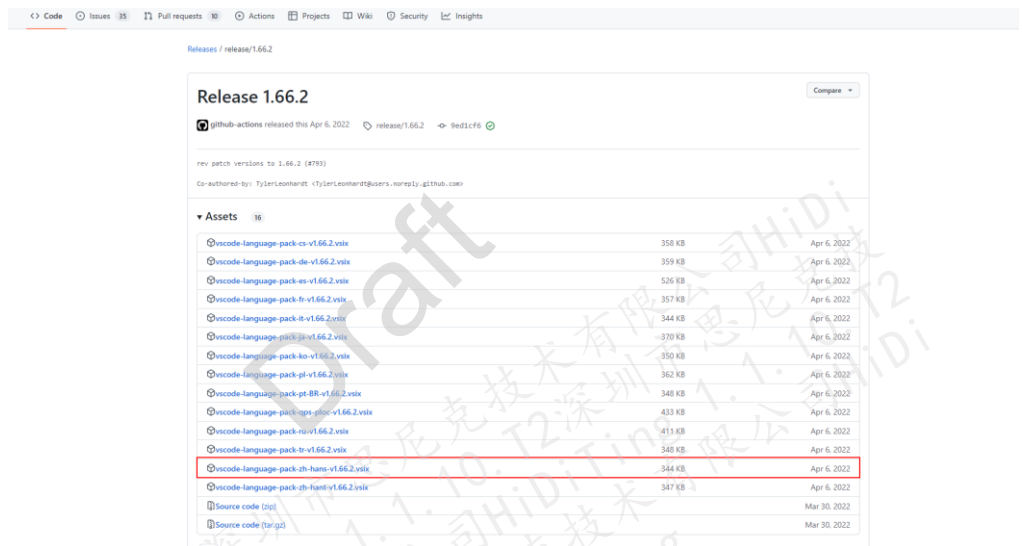
2. 从弹出的页面中查看 Visual Studio Code 版本。



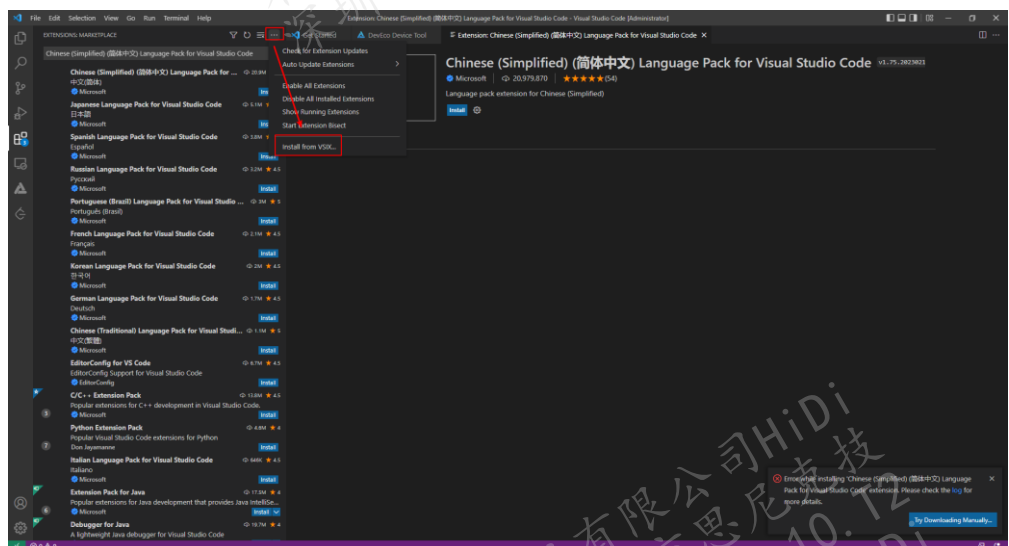
3. 打开 [Tags · microsoft/vscode-loc · GitHub](#) 网站，找到与 Visual Studio Code 对应版本号的 tag 并点击。



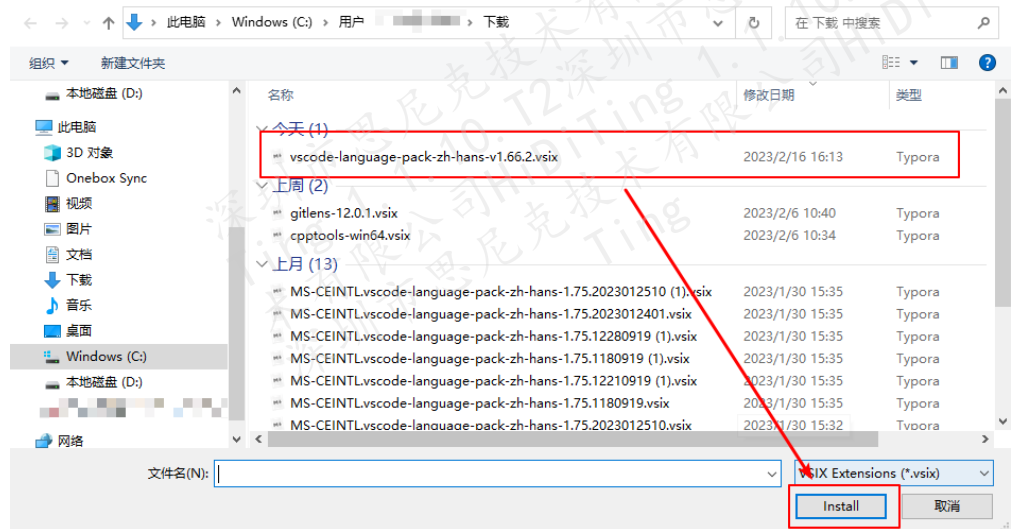
4. 点击名称中带有“hans”字样的插件，浏览器会自动下载插件。



5. 回到 Visual Studio Code 中，点击“...”，再点击“Install from VSIX”，打开选择插件对话框。



6. 选择刚才下载的插件，点击 “Install” 。



7. 安装过程较快，大概几秒钟就能安装完成。安装完成后会在右下角弹出安装成功和重新启动通知，点击 “Restart” 按钮，Visual Studio Code 会重新启动，此时 Visual Studio Code 已变为中文界面。

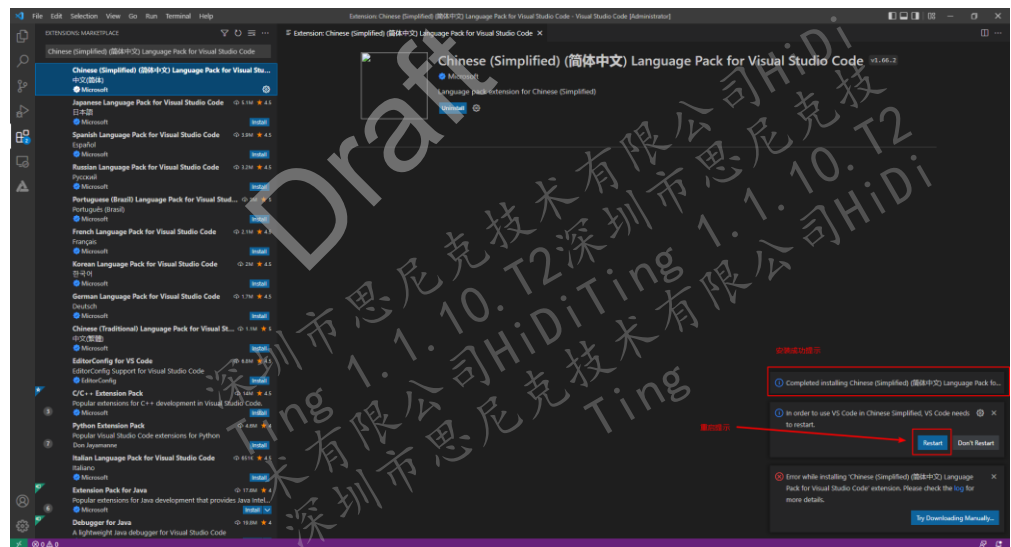
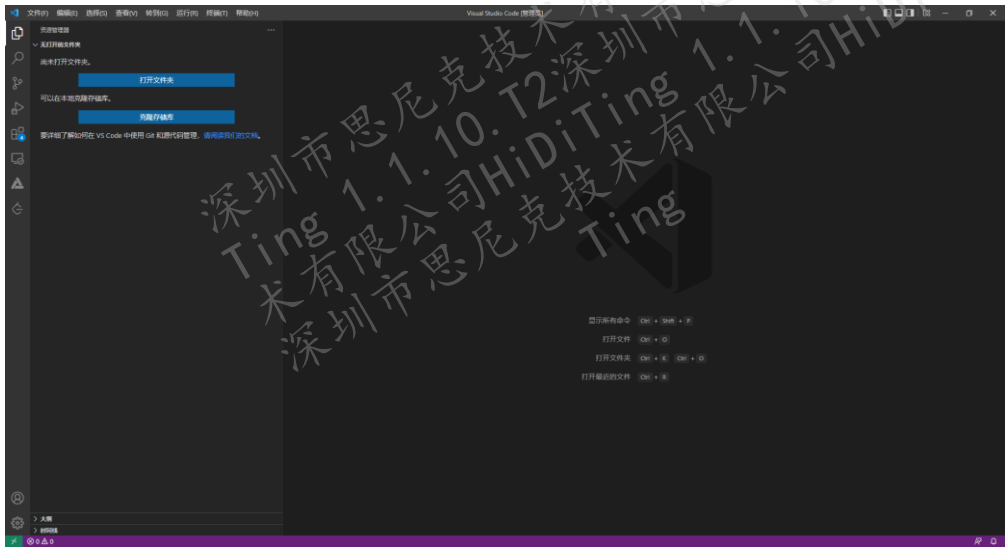
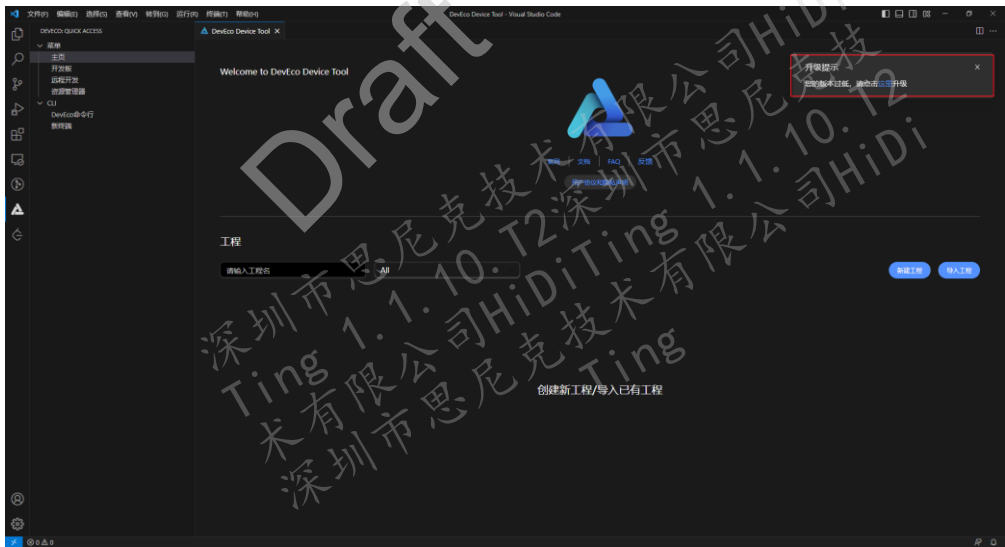


图2-3 Visual Studio Code 重启后界面



DevEco Device Tool 工具界面（右上角会出现“升级提示”，用户无需理会，也不要升级）。



-----结束

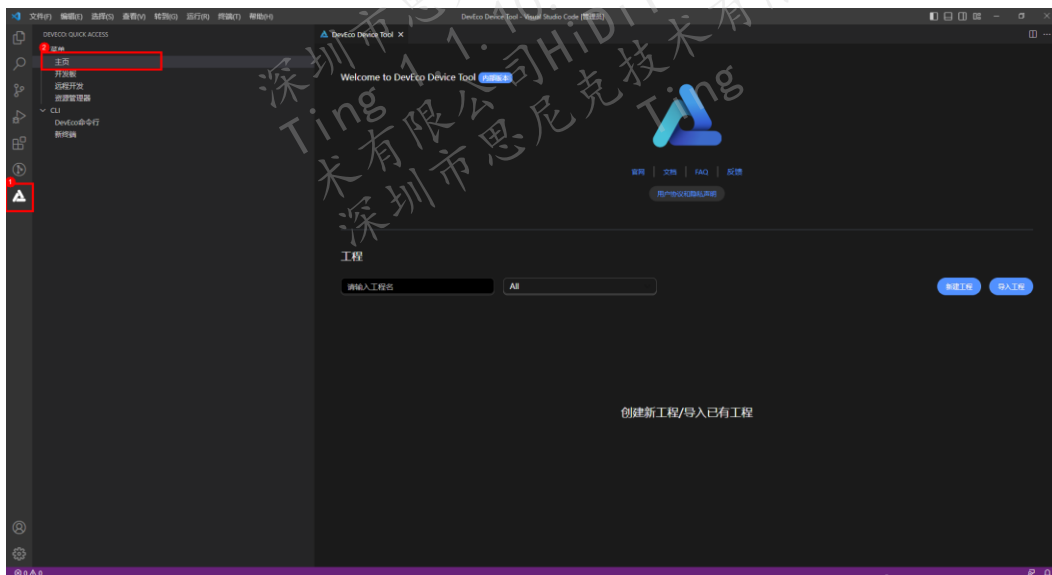
3 工程管理

本章节主要介绍在获取 BrandyK100 SDK 后，如何使用 DevEco Device Tool 工具导入开发工程，以便于进行后续编译、烧录等操作（由于 Windows 系统限制，在解压工程包时应尽量缩短存储目标的路径长度，路径内不可带有空格）。

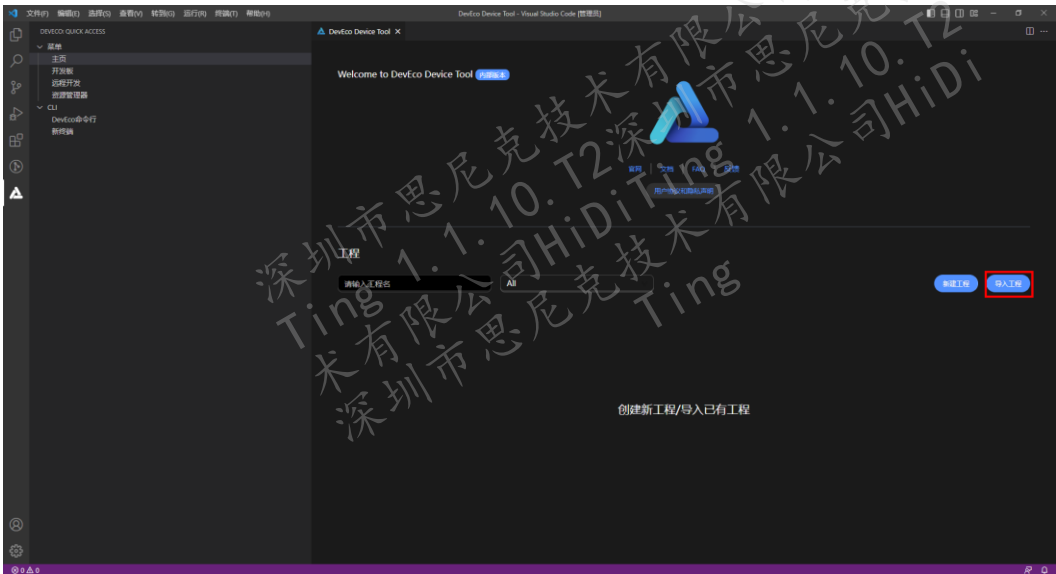
须知

目前工程配置中支持符号范围为[0~9 A~Z a~z _ - . :]，因此在导入路径时，应避免采用其他特殊字符，否则会导致在工程配置时，路径选择失败。

步骤 1 打开 DevEco Device Tool 主页。打开 VsCode 时会自动弹出 DevEco Device Tool 界面，若未弹出或关闭，可点击左侧侧边栏的工具按钮，再点击主页，弹出界面。



步骤 2 在右侧页面中点击导入工程按钮。



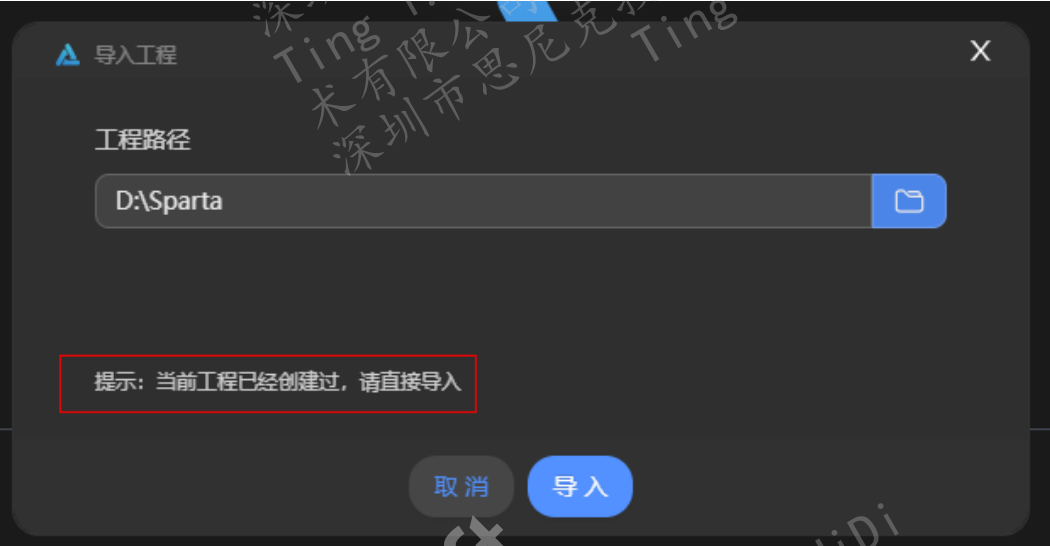
步骤 3 在导入工程弹窗中选择解压出来的 BrandyK100 SDK 工程路径，点击“导入”。



步骤 4 选择 SOC 为“BRANDY”，开发板和框架会根据 SOC 进行自动生成，无需人工选择。然后点击“导入”按钮。



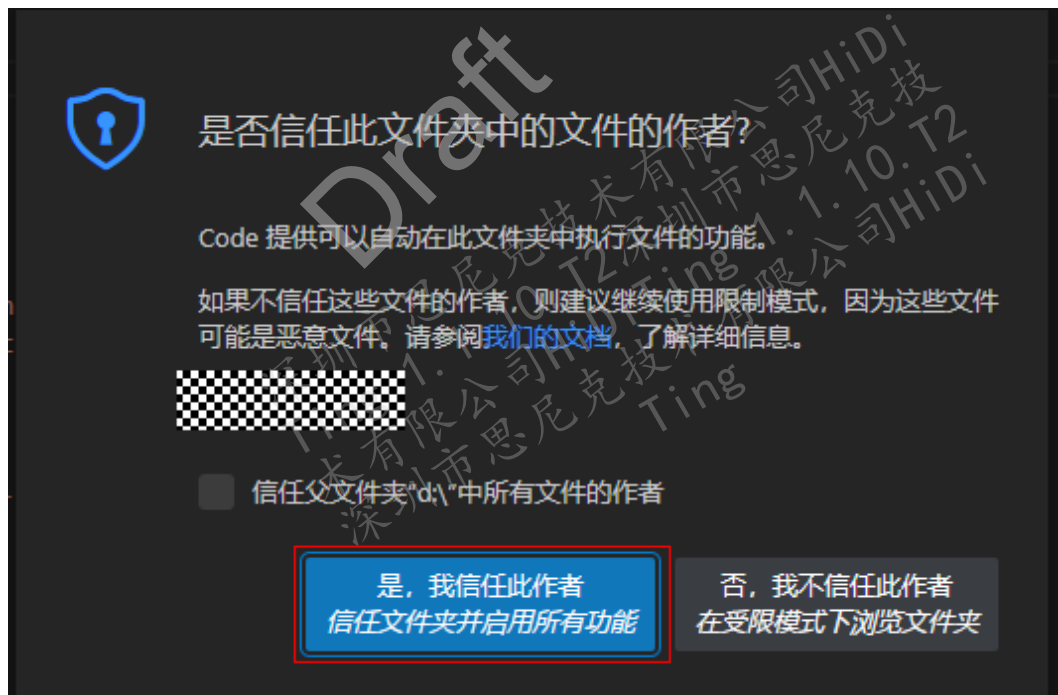
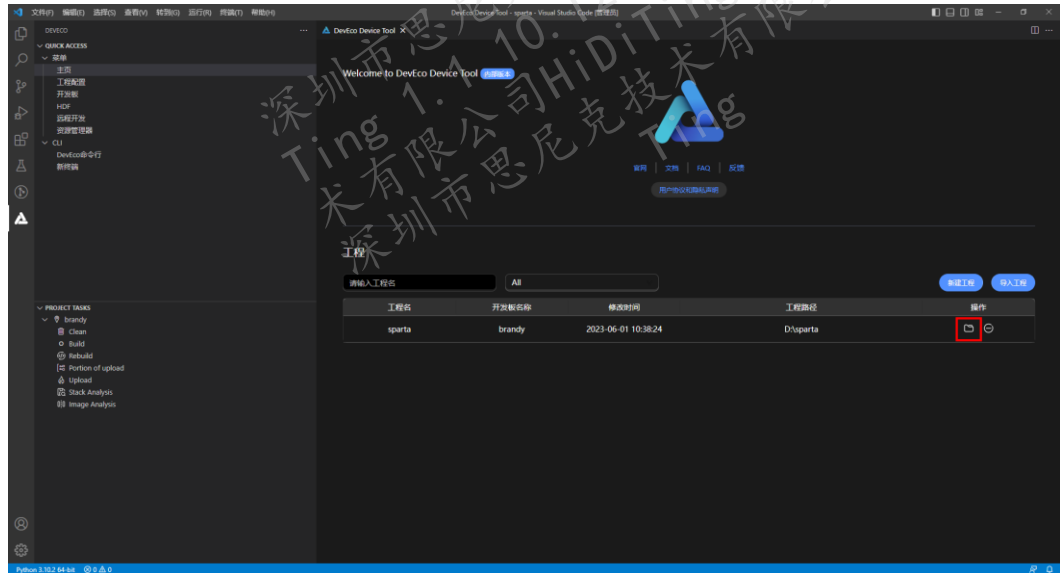
如果待导入的目录不是第一次导入，则导入时会显示导入失败，并提示“当前工程已经创建过，请直接导入”，此时直接点击“导入”按钮即可，若出现“工程已打开，请选择其它工程”，说明该工程已经在本工具中打开，点击取消即可进入[步骤 5](#)。若想重新导入工程，可以手动删除在 SDK 工程路径下生成的.vscode 和.deveco 文件，再导入工程，即可实现重新导入。



电脑 > 本地磁盘 (D:) > Sparta

名称	修改日期	类型	大小
.deveco	2023/2/16 15:06	文件夹	
.vscode	2023/2/16 15:06	文件夹	
doc	2023/2/16 14:46	文件夹	
software	2023/2/16 14:46	文件夹	
tools	2023/2/16 14:50	文件夹	
.gitignore	2023/2/16 15:06	文本文档	1 KB

步骤 5 工程导入成功后，若弹出是否信任此文件夹中的文件的作者，可选择信任此作者以启用所有功能。后续可在 DevEco Device Tool 工具主页直接打开已导入成功的工程，再点击左侧插件按钮跳转到工程界面。



说明

若出现打开已有工程时左下角 task 列表未正常显示，可尝试删除配置文件后重新导入该工程，详细操作过程可参照上述步骤 4，关闭 VSCode 工程窗口后，删除 SDK 路径下的 ".deveco" 文件夹再重新导入该 SDK。

-----结束

4 编译运行

4.1 编译 BrandyK100 工程

4.2 自定义编译命令方法介绍

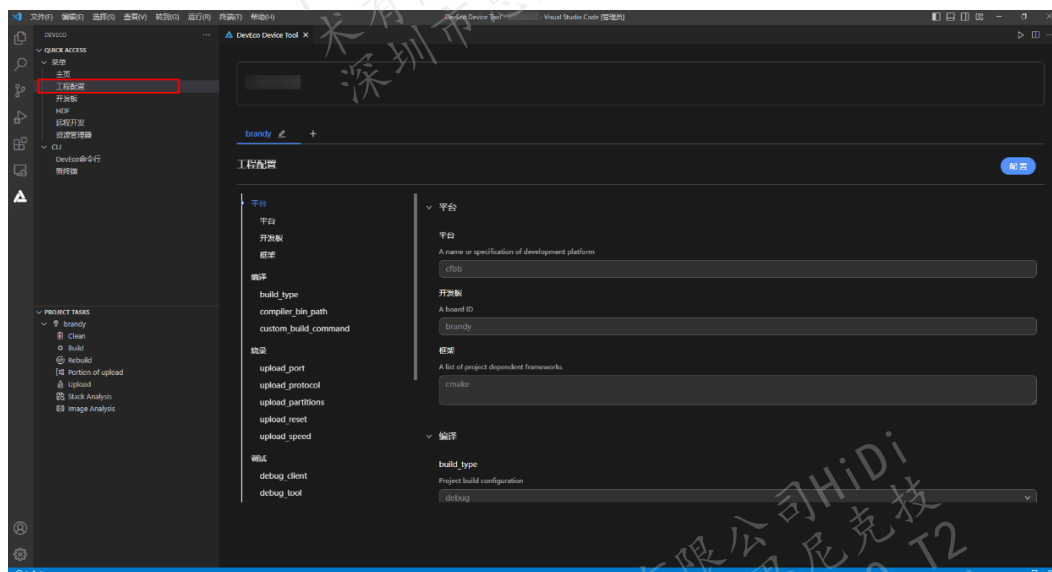
4.1 编译 BrandyK100 工程

本小节主要介绍如何使用 DevEco Device Tool 工具编译 BrandyK100 工程，编译 BrandyK100 工程方法如下。

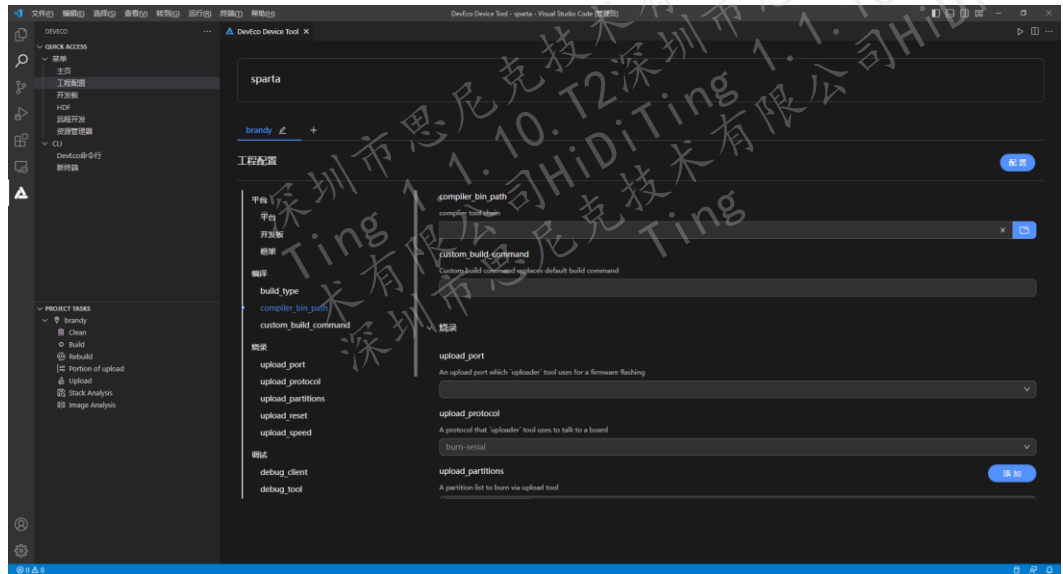
说明

默认编译 target: brandy-target3, 若需要编译其他 target 请参见“4.2-自定义编译命令方法介绍”章节。由于 Windows 有路径长度限制问题（绝对路径小于 260 个字符），请控制 SDK 根路径长度以保证编译正常运行。

步骤 1 完成导入工程或打开原有工程后，点击菜单栏中的工程配置按钮，打开工程配置。



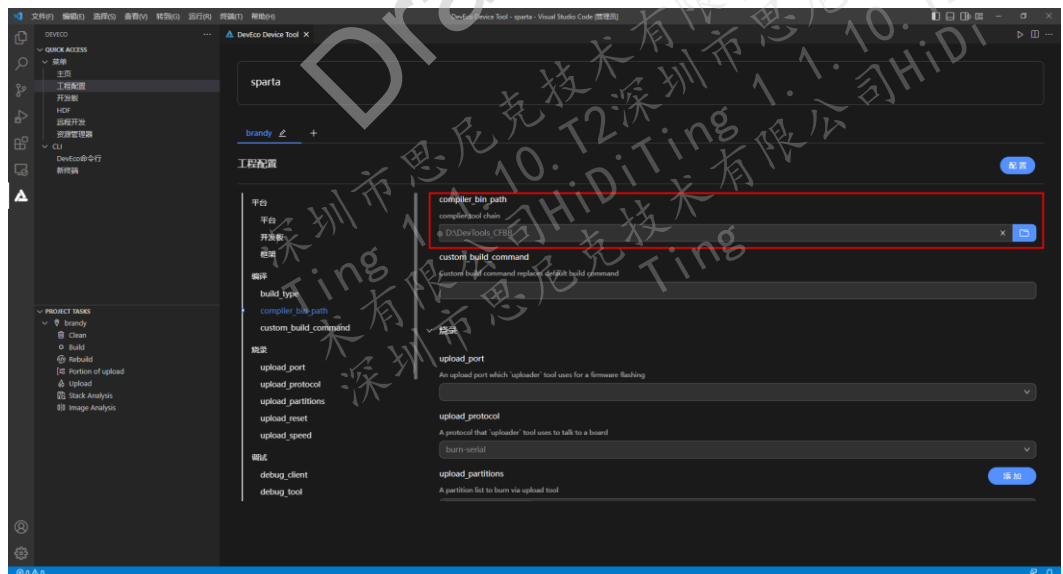
步骤 2 点击工程配置中的“compiler_bin_path”按钮。



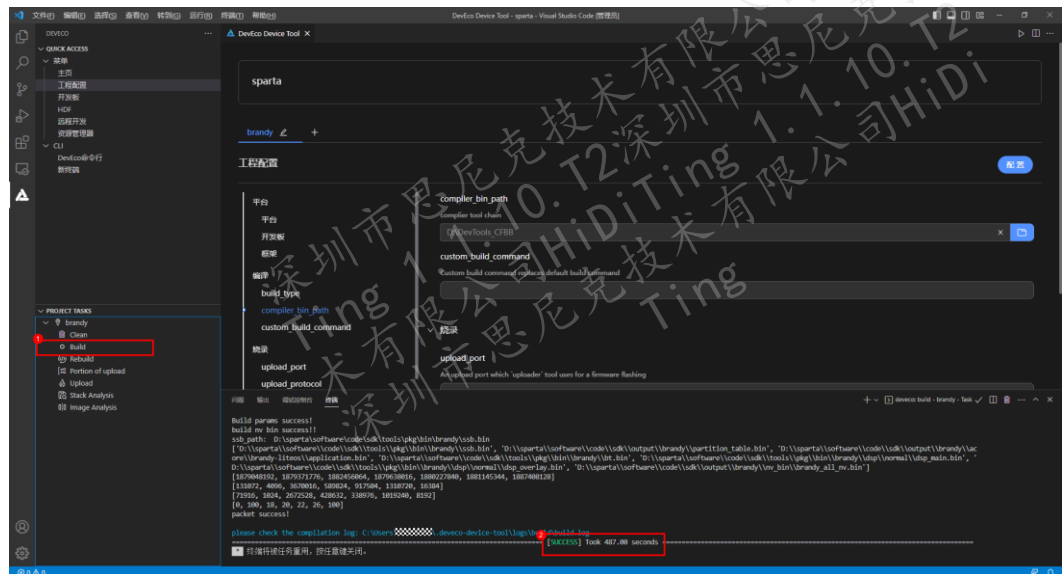
步骤 3 点击文件夹按钮设置“compiler_bin_path”为 DevTools_CFBB 工具包（在提供 SDK 时会提供该工具，默认路径在 SDK 解压后的“tools/DevEco”路径下）。

说明

“compiler_bin_path”目录中不能包含中文字符。



步骤 4 点击 PROJECT TASKS 菜单中的“Build”按钮，进行编译 BrandyK100 工程。第一次编译大概在需要 400 秒到 1000 秒，请耐心等待完成。编译完成之后，工具会自动将编译结果中的 bin 文件进行打包，“SUCCESS”代表编译成功和打包成功。打包好的 fwpkg 文件在“software\code\sdk\tools\pkg\fwpkg\brandy”文件夹中。



说明

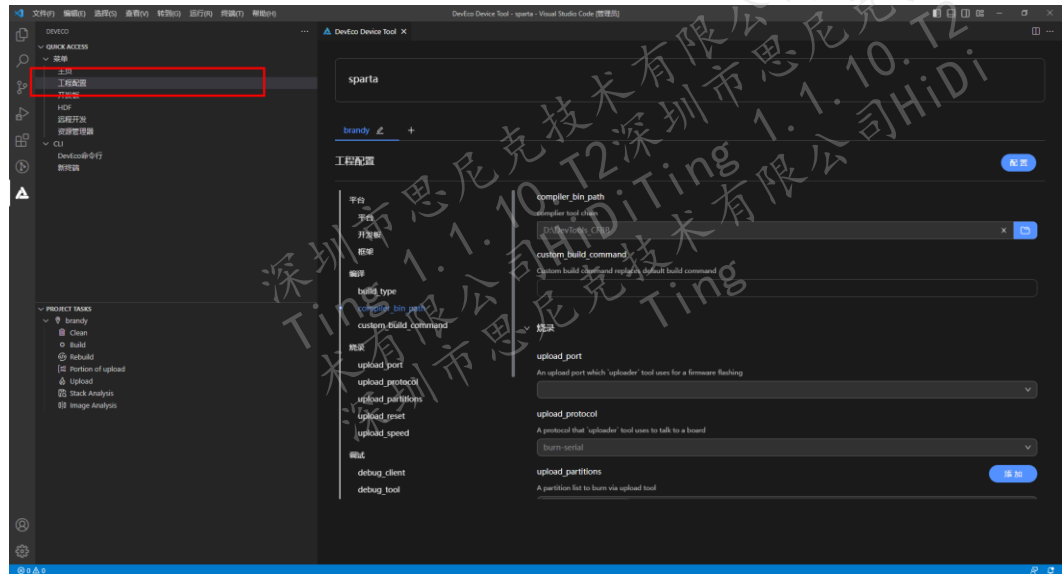
目前编译按照 CPU 最大线程数执行，可能会导致 CPU 占用率过高、电脑卡顿等性能问题，若需要降低 CPU 占用率，请手动输入“custom build command”（请参见“4.2 自定义编译命令方法介绍”）进行控制，参数为-jx，将编译使用的线程数限制为 x，例如：brandy-target3 -j8 将线程数限制为 8。限制线程数的同时，有可能会延长编译时间，请视实际需要使用。

-----结束

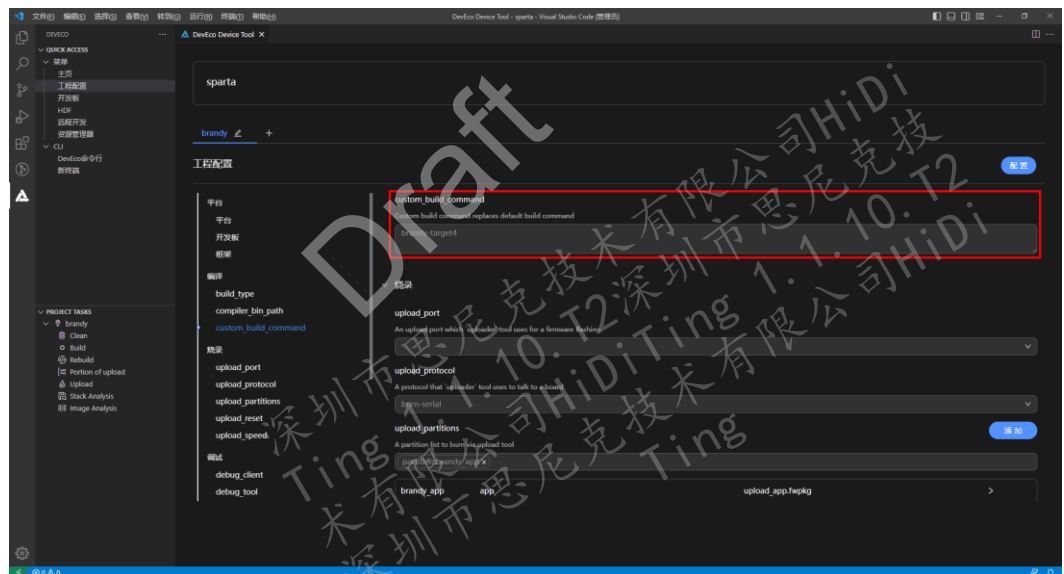
4.2 自定义编译命令方法介绍

在工具中点击“Build”按钮后，会自动调用“brandy-target3”命令进行编译。如果想调用其他编译命令（其他 target 可在“software\code\sdk\build\config\target_config\brandy\config.py”中查看名称及其配置），可以根据如下方法进行操作：

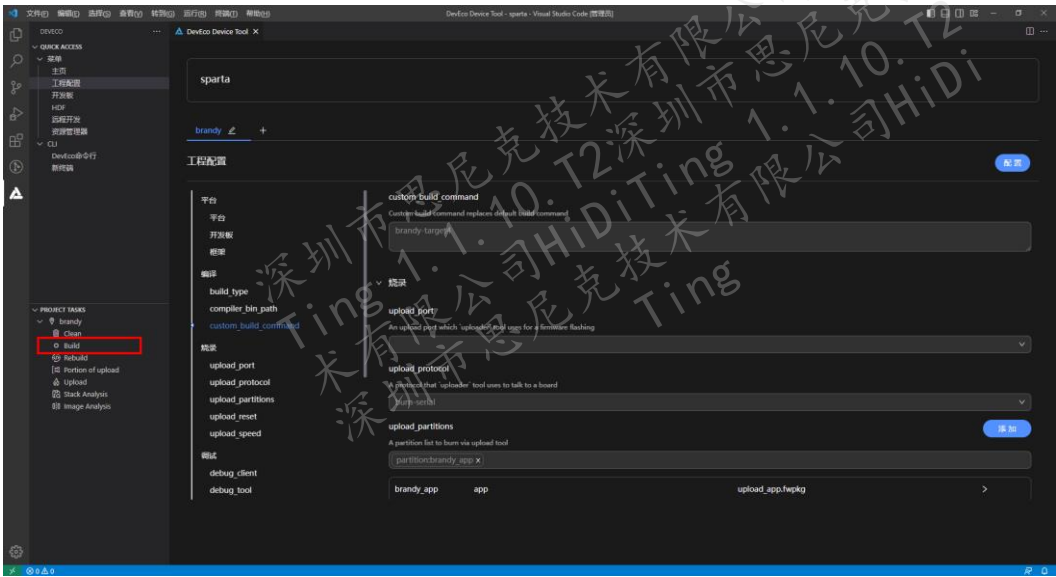
步骤 1 点击菜单中的“工程配置”按钮。



步骤 2 在工程配置界面找到“custom_build_command”设置，输入想要配置的编译命令，例如：brandy-target4，即可编译 target 为 brandy-target4 的内容。



步骤 3 点击功能区中的“Build”按钮，开始编译。



-----结束

5 软件包烧录

如果已经按照此文档操作过“[4.1 编译 BrandyK100 工程](#)”，则可以跳过“[5.1 烧录工具配置](#)”。

[5.1 烧录工具配置](#)

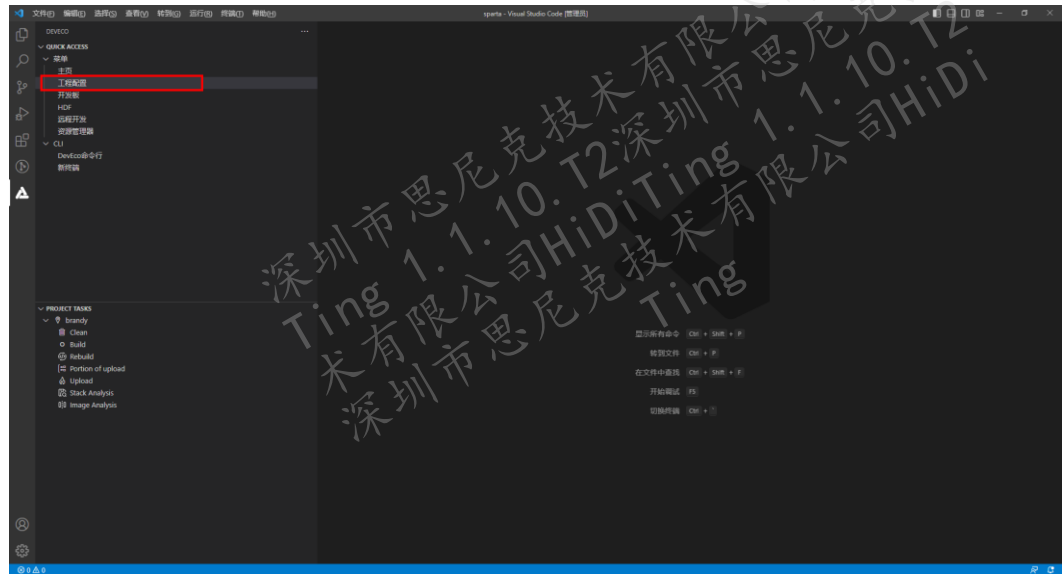
[5.2 烧录工程配置](#)

[5.3 执行工程烧录](#)

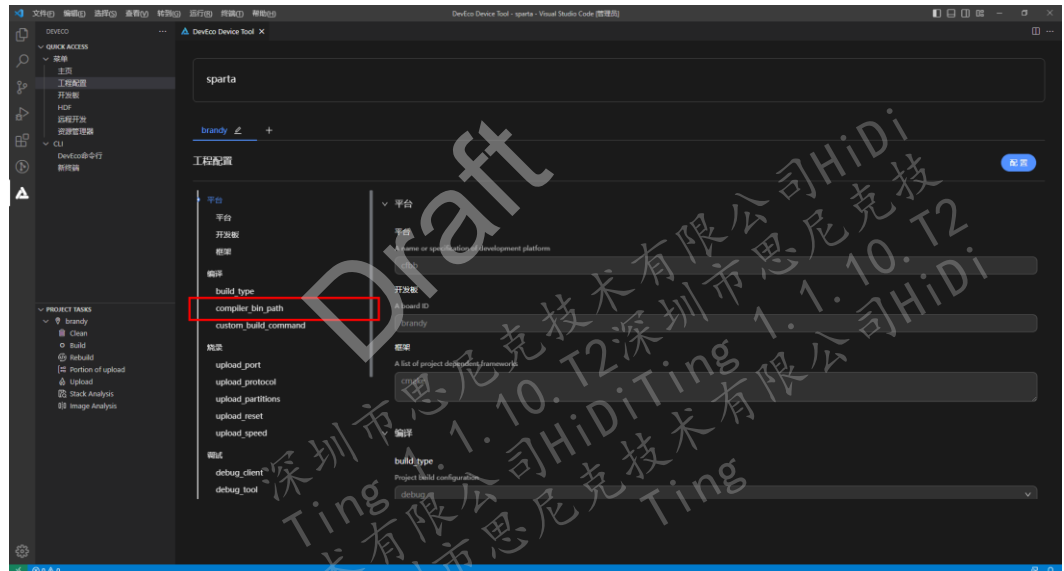
5.1 烧录工具配置

当前 DevEco Device Tool 工具支持 BrandyK100 单板一键烧录功能，该功能依赖于 BrandyK100 交付件中 DevTools_CFBB 工具包，需要用户在 DevEco Device Tool 工程配置中手动配置“compiler_bin_path”路径。具体配置方法如下：

步骤 1 点击菜单栏中的“工程配置”按钮，打开工程配置。



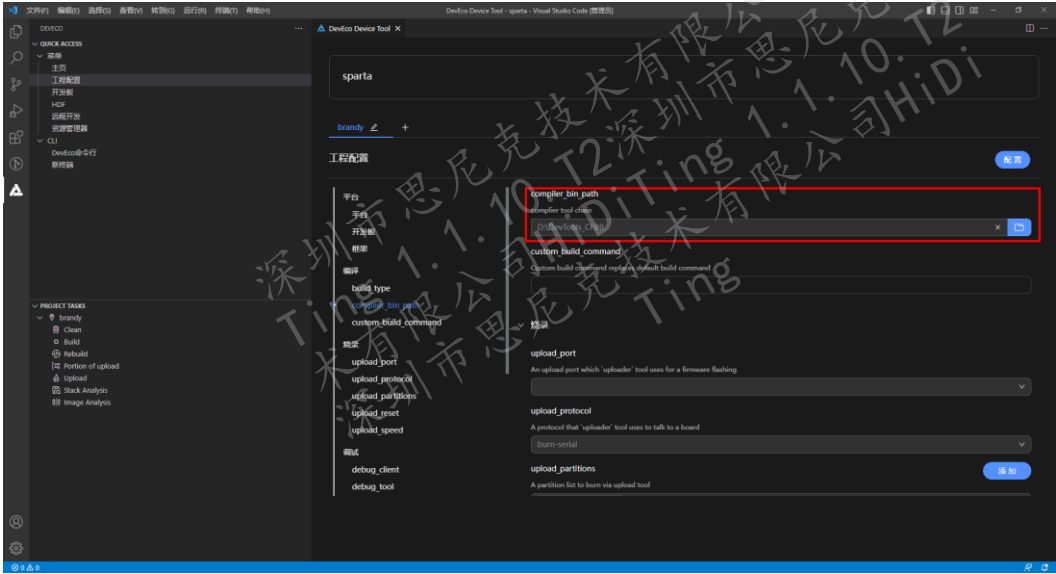
步骤 2 点击工程配置中的“compiler_bin_path”按钮。



步骤 3 点击文件夹按钮设置“compiler_bin_path”为 DevTools_CFBB 目录。

说明

“compiler_bin_path”目录中不能包含中文字符。

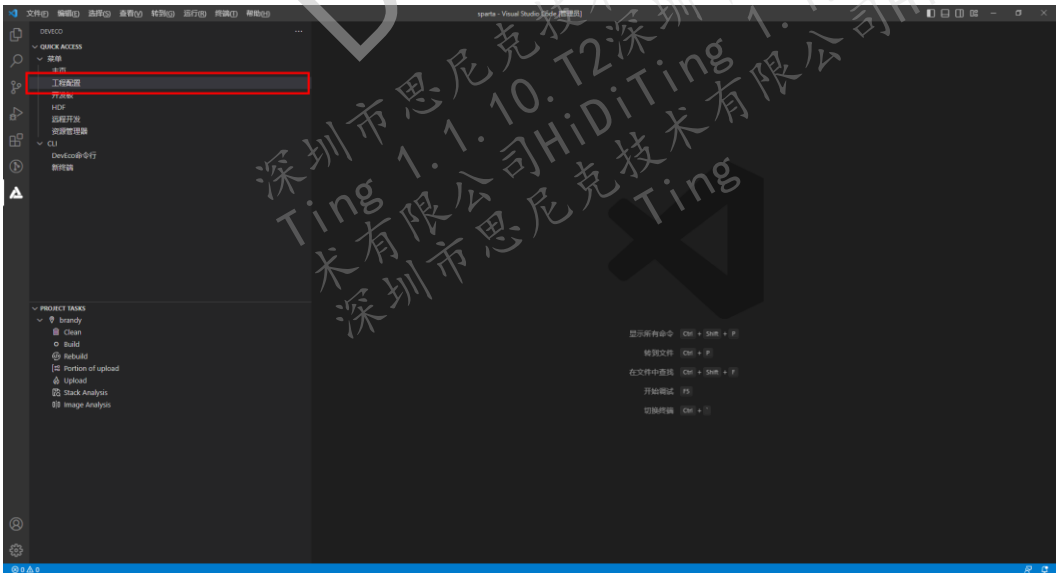


----结束

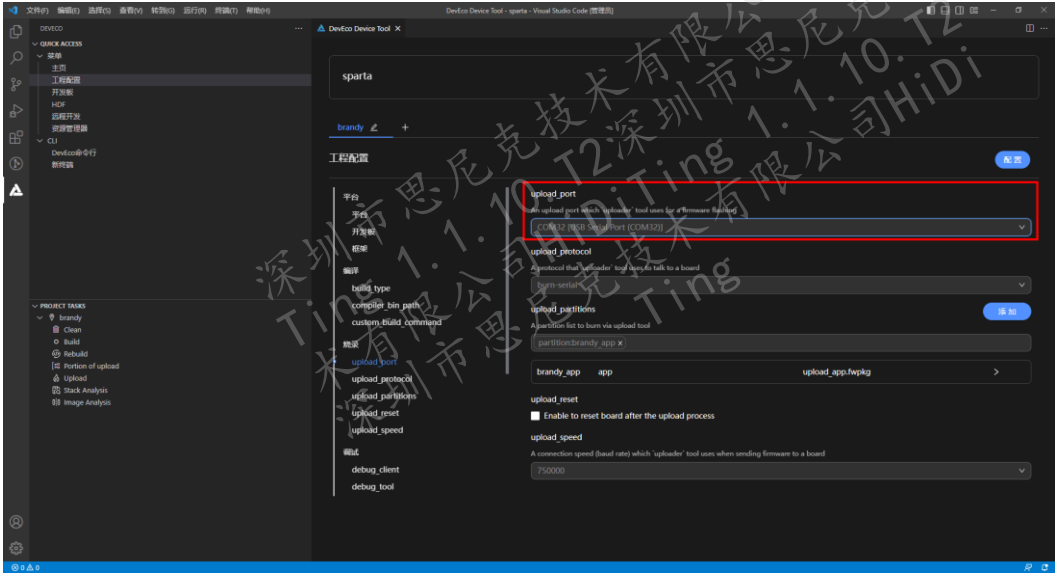
5.2 烧录工程配置

烧录工程配置方法如下：

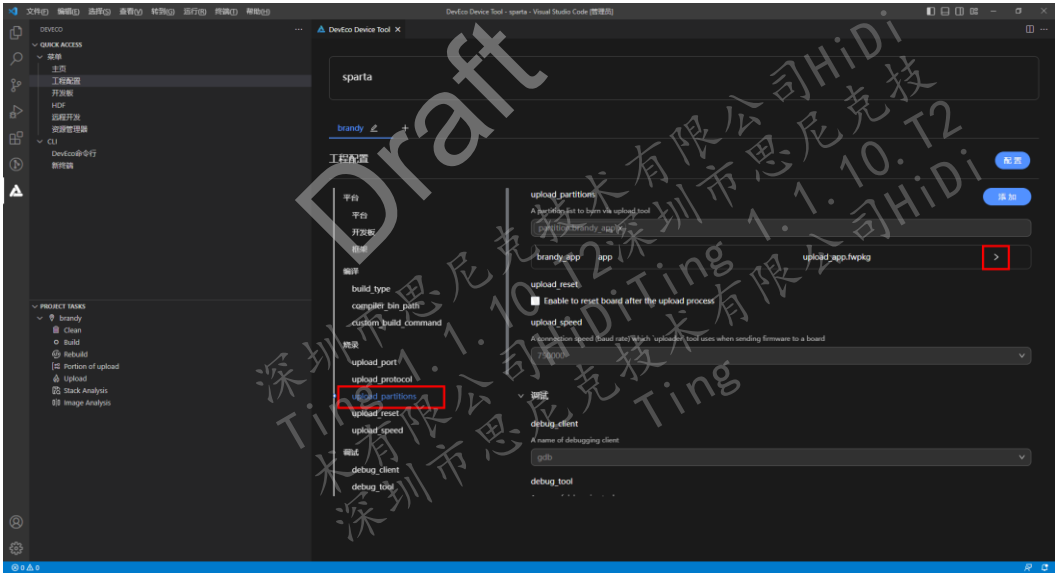
步骤 1 点击基本功能区中菜单栏的“工程配置”按钮进入工程配置。

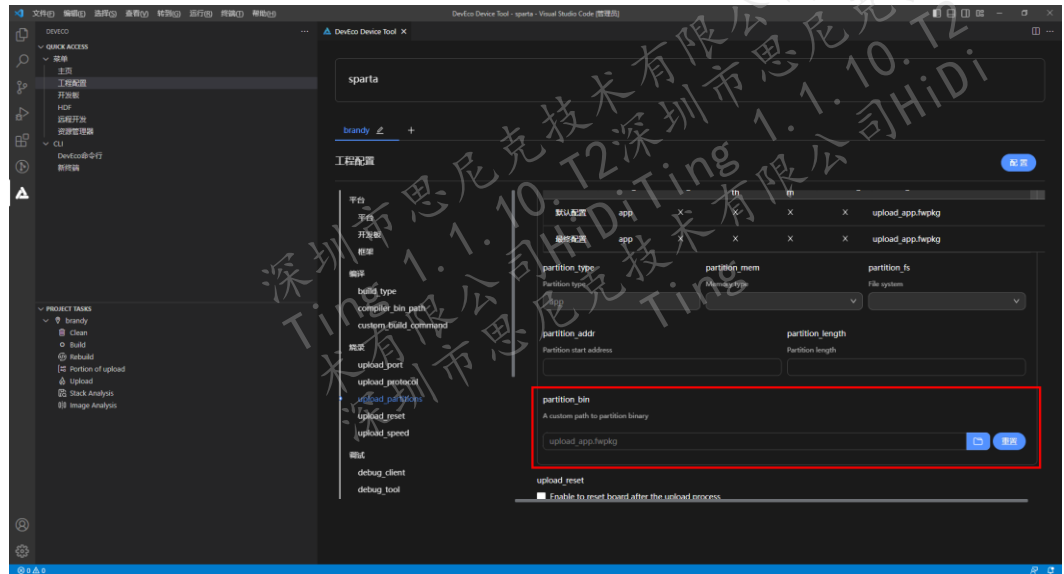


步骤 2 点击“upload_port”按钮设置烧录的串口。

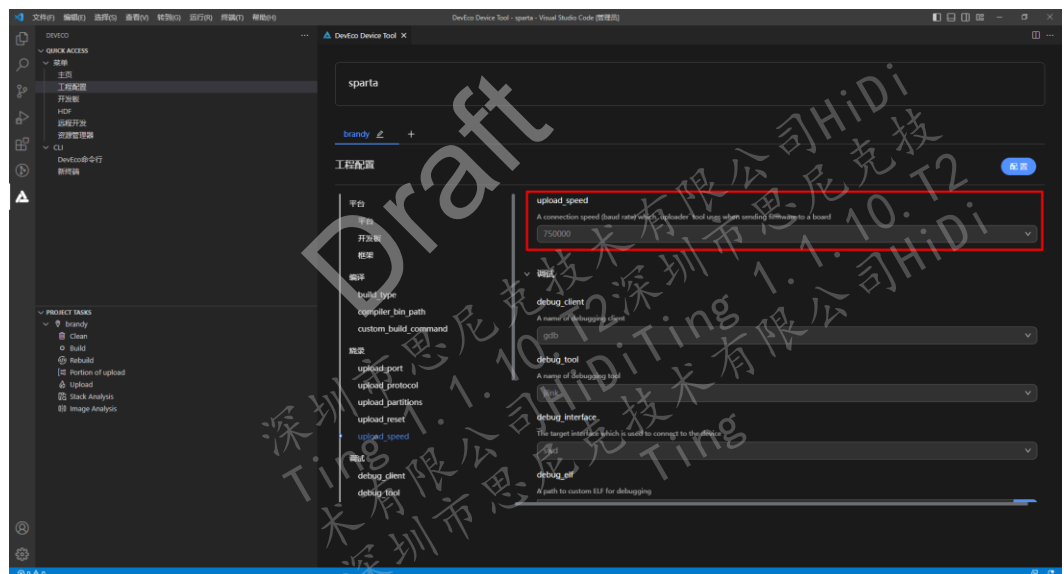


步骤 3 依次点击“upload_partitions”按钮、“>”图标，然后下滑页面，会看到选择文件按钮，点击选择文件按钮选择需要烧录的文件。（已完成自适应默认配置，该步骤非必须）





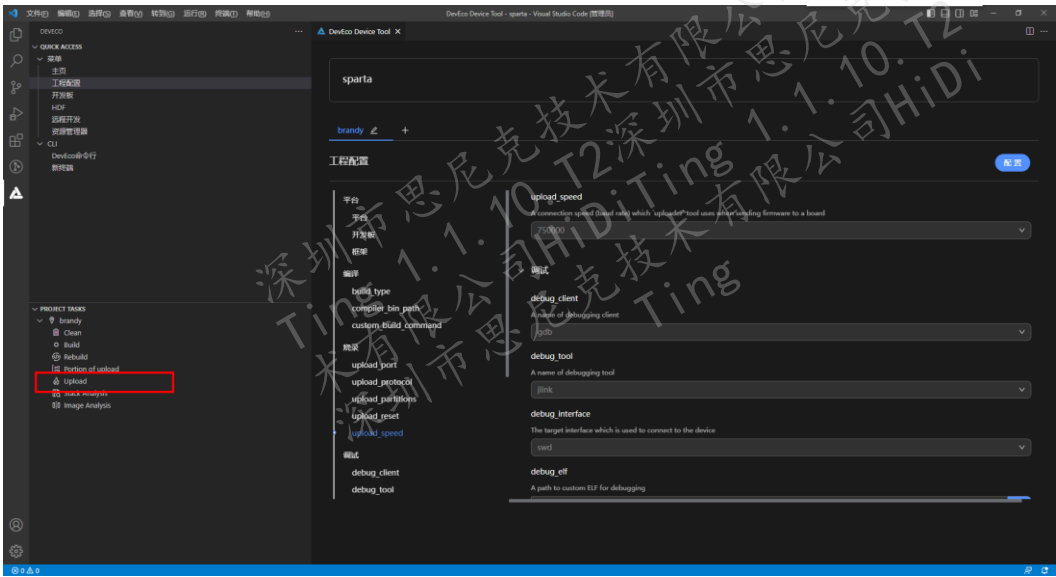
步骤 4 点击“upload_speed”按钮设置烧录波特率，默认为 750000（允许的烧录速率范围 115200 ~ 2000000）。



----结束

5.3 执行工程烧录

步骤 1 点击左侧开发板任务栏 Upload 按钮开始烧录。



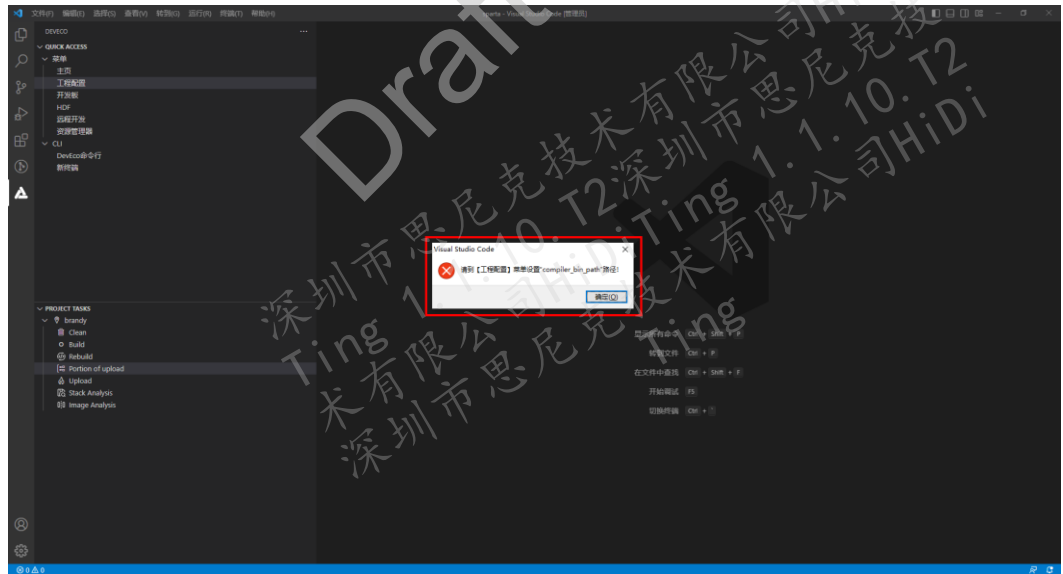
- 步骤 2 点击“Upload”后，150s 内需要手动复位单板，方可开始烧录。
- 步骤 3 烧录进度达到 100%后，表示烧录完成（烧录完成后单板会进行自动重启）。



----结束

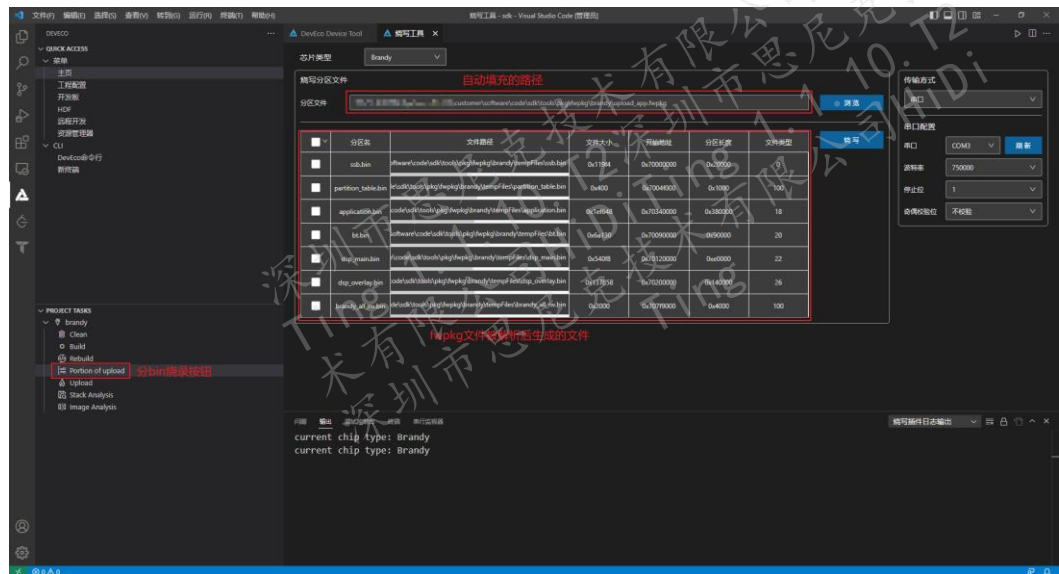
6 分 bin 烧录

本章节主要讲解用户如何只烧录单个 bin 文件的方法。使用此功能的前提是需要配置“compiler_bin_path”路径，如果已经配置过此项，则直接从本章节[步骤 1](#)开始，如果未配置此项，需要先阅读[5.1 烧录工具配置](#)（在设置“compiler_bin_path”值之后，如果点击“Portion of upload”按钮出现下图提示，原因为设置“compiler_bin_path”值之后系统任务更新需要 2~3 s，等待 3 秒后再点击此按钮，即可正常使用此功能），然后再阅读此章节后续部分。

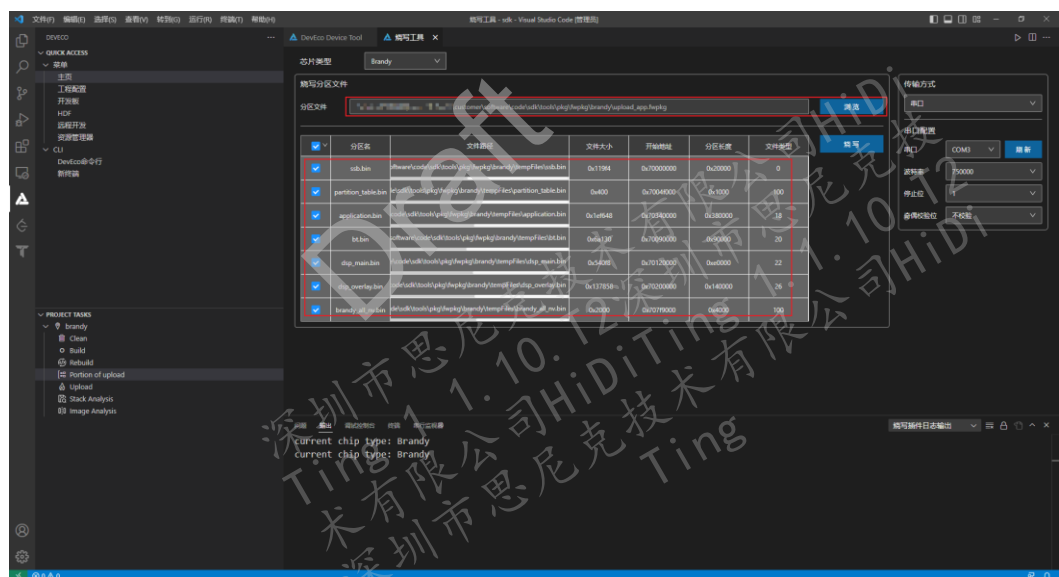


烧录单个 bin 文件的方法如下：

- 步骤 1 点击 PROJECT TASKS 中的“Portion of upload”按钮，DevEco Device Tool 工具会调用烧写工具并在右侧展开。默认情况下，烧录工具会在工程目录下查找 fwpkg 文件，自动填充到分区文件中；填充之后烧录工具会自动解析 fwpkg 文件，并将解析的文件展示在页面上。



步骤 2 如果需要更换 fwpkg 文件，可以点击“浏览”按钮从本地文件中选择打包好的 fwpkg 文件。选择完成后，烧录工具会自动的将 fwpkg 文件中包含的 bin 内容列出。



步骤 3 勾选需要烧录的 bin 文件。需要注意的是，如果此次烧录只是烧录一个 bin 文件或几个（并不是全部都烧录），那么需要保证 BrandyK100 板为非空板。如是空板，那么需要先执行一次全 bin 烧录，再进行分 bin 烧录。

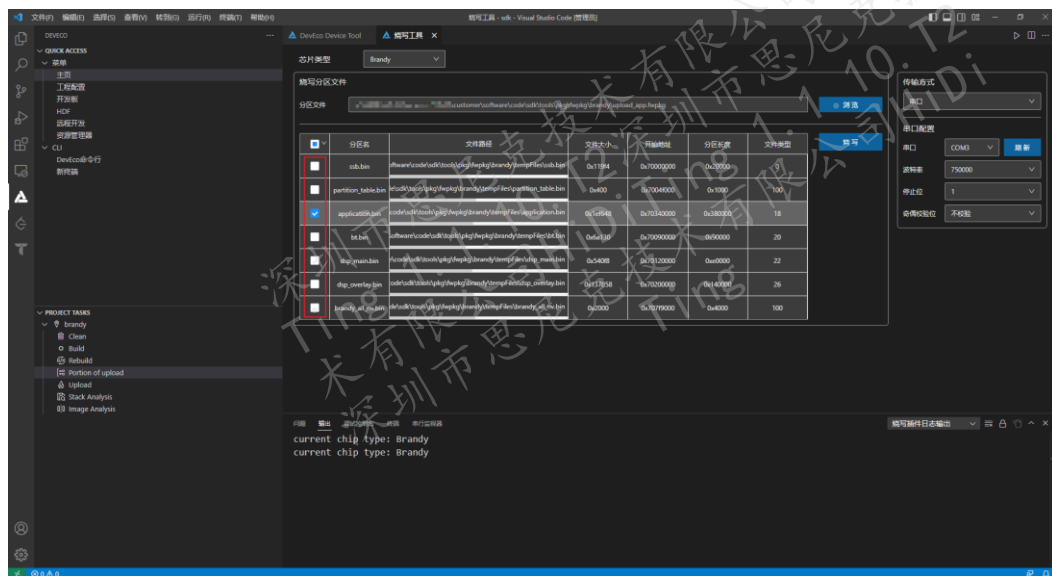
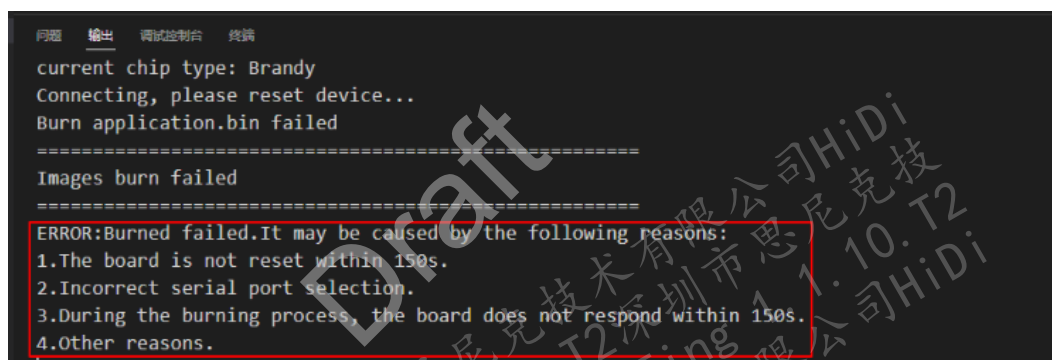
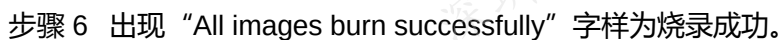
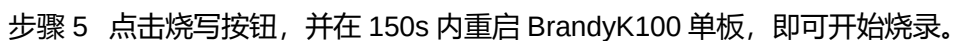
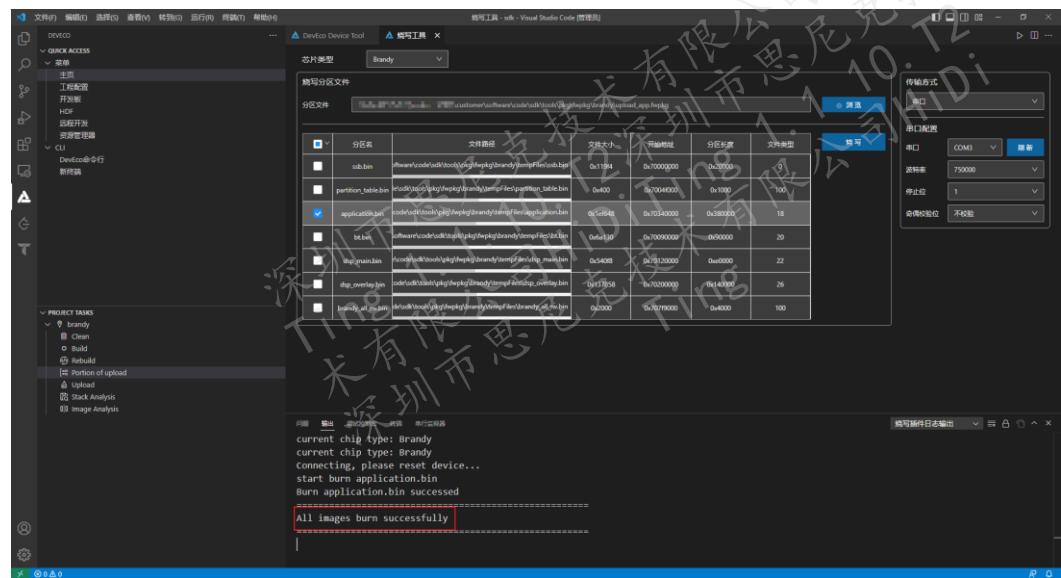


图6-1 空板烧录失败



步骤 4 配置串口信息，设置烧录串口和波特率。波特率推荐选择 750000（单板支持的波特率范围是 115200 ~ 2000000）。





----结束

7

栈分析和镜像分析

DevEco Device Tool 集成了 Stack Analysis 栈分析工具和 Image Analysis 镜像分析工具，用于对开发过程中的内存不足、内存溢出等问题进行分析，帮助开发者更加精准的定位问题。

- Stack Analysis 栈分析工具是基于静态二进制分析手段，提供任务栈开销估算值和函数调用关系图示，为栈内存使用、分析、优化、问题定位等开发场景提供较为准确的静态内存分析数据参考。
- Image Analysis 镜像分析工具对工程构建出的 elf 文件进行内存占用分析，支持开发者快速评估内存段、符号表使用情况。

7.1 路径配置

7.2 栈分析

7.3 镜像分析

7.1 路径配置

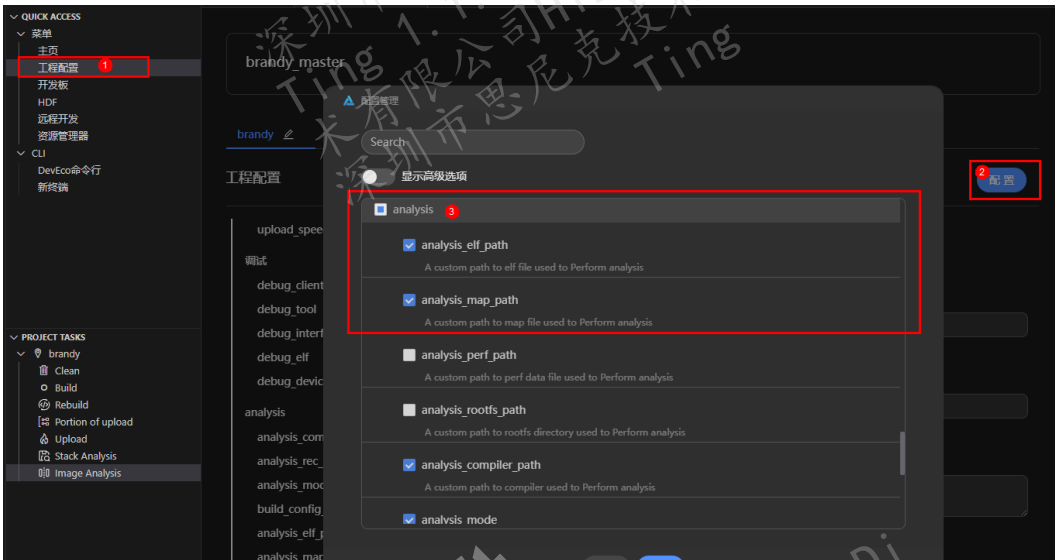
在使用工具的栈分析和镜像分析时需要使用到 elf 文件、map 文件以及编译工具链，其中 elf 文件和 map 文件需要编译 sdk 得到，编译工具链包含在 sdk 文件夹中。在直接导入发布包情况下，无需用户配置，DevEco Device Tool 工具会自动使用 output 目录下编译生成的 elf 文件和 map 文件进行栈分析和镜像分析。若同一个工程编译了多个 target，则需要指定 elf 文件和 map 文件。模块配置文件路径、栈分析分析模式和递归循环次数配置方法如下：

- 步骤 1 打开工程配置，如果工程配置面板没有 elf 文件路径 “analysis_elf_path” 和 map 文件路径 “analysis_map_path” 设置项，则要在配置管理窗口中打开这两个配置项：单击“配置”按钮打开配置管理窗口，在配置管理窗口中找到 “analysis_elf_path” 和

“analysis_map_path” 并勾选，点击配置管理窗口上的确定按钮后，退出配置管理窗口。

说明

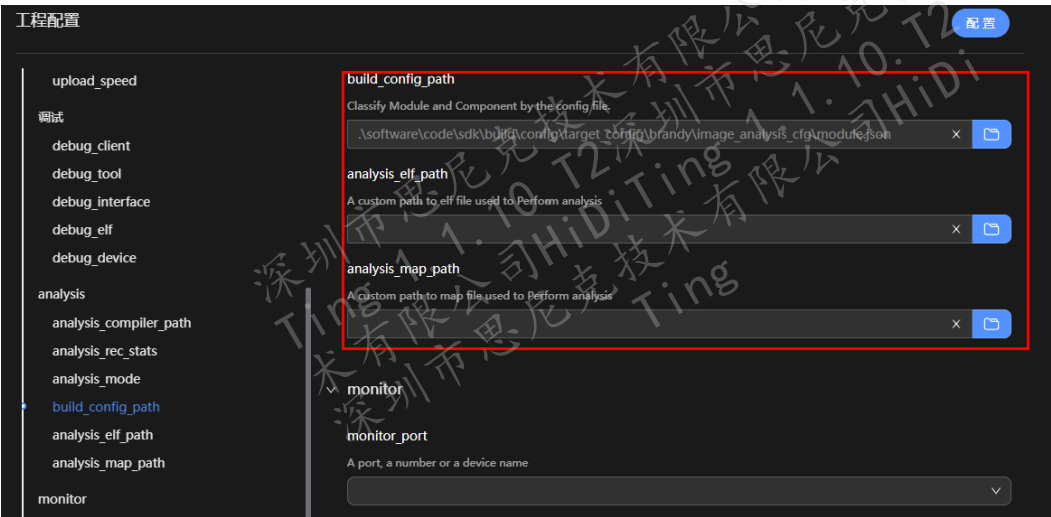
如果工程配置页面中有对应的参数配置项，则可省略此步骤。



步骤 2 选择导航栏的 analysis 模块，定位到分析参数的配置区域。



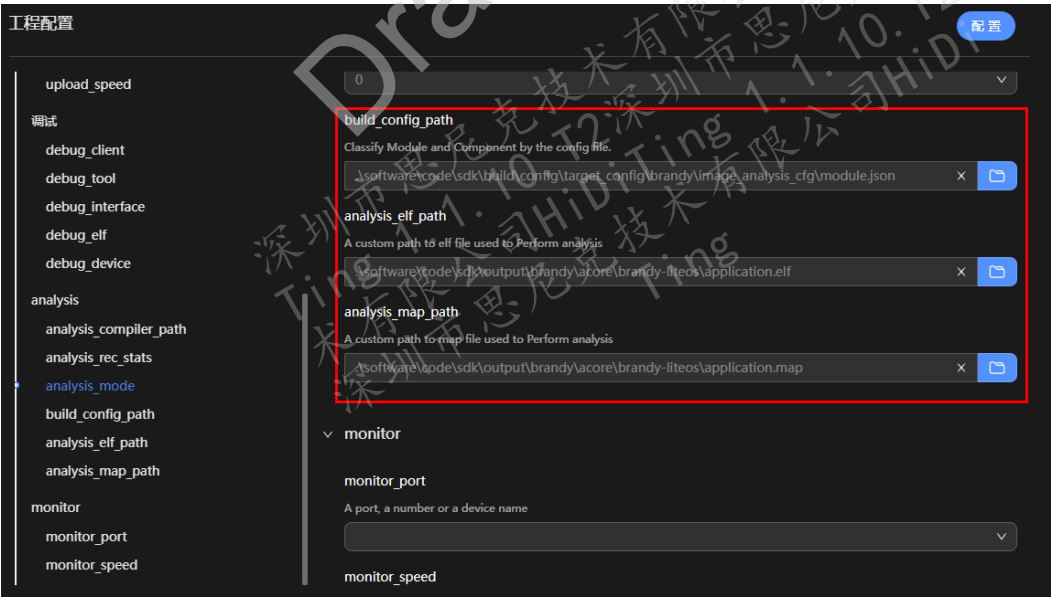
步骤 3 选择模块配置文件、elf 文件、map 文件和编译工具链目录：



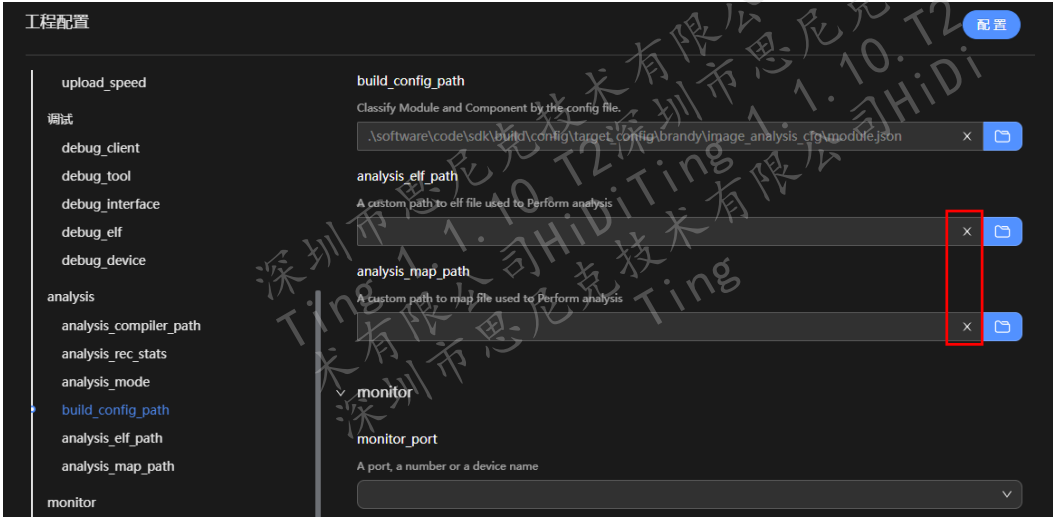
模块配置文件中规定了 SDK 工程目录中每个目标文件对应的模块名称，默认路径为“\software\code\sdk\build\config\target_config\brandy\image_analysis_cfg\module.json”。

elf 文件和 map 文件的路径默认为空，有自适应匹配功能，IDE 根据不同的编译 target 自动找寻 elf 文件和 map 文件。

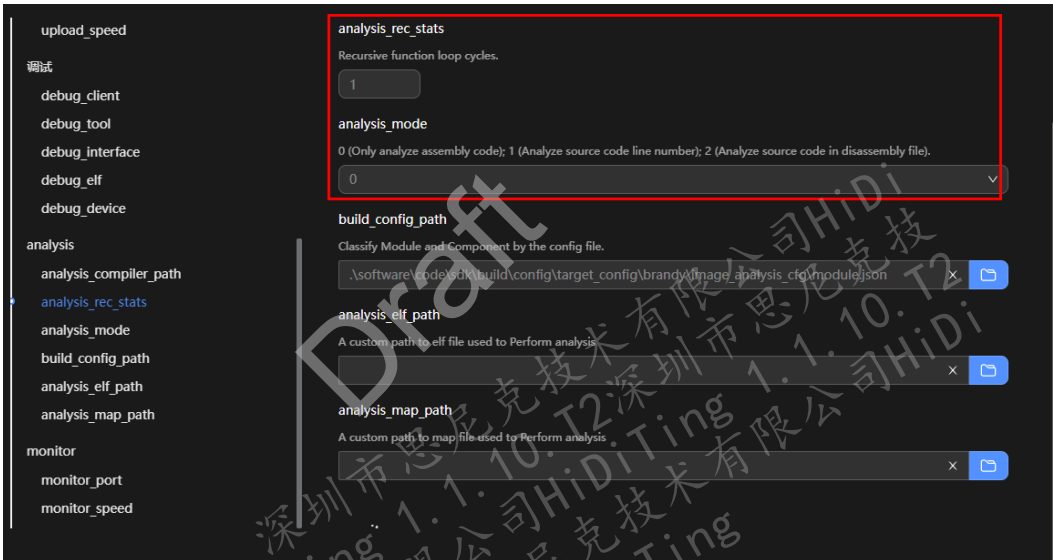
用户也可以指定 elf 文件和 map 文件，关闭自适应匹配：



按下 “x” 图标可清空输入的路径，恢复自适应匹配路径的功能：



步骤 4 配置递归函数的循环次数和分析模式。



当不设置循环次数或者分析模式时，栈分析使用默认参数进行分析，默认递归函数循环为 1 次，分析模式为模式 0，配置项说明如下。

配置项说明：

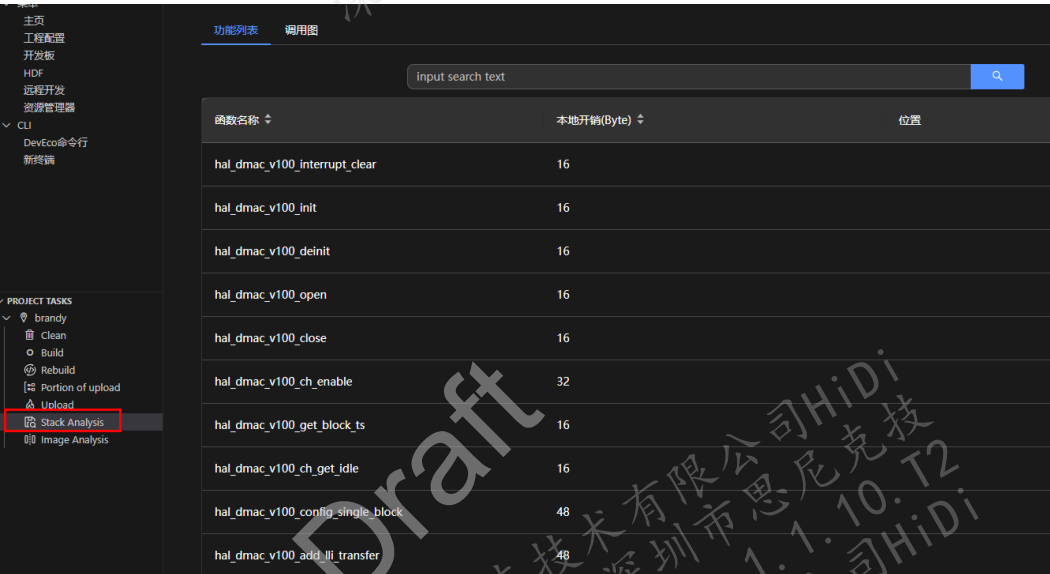
- 循环调用次数的栈分析入参：循环次数影响最大开销的统计，默认值为 1，取值范围最小为 0 次，最大为 4096 次。最大开销统计值为当前函数所有子函数中最大的开销与循环调用的次数的乘积再加自身的开销。
- 栈分析模式选择：
 - 模式 0：分析反汇编文件，统计函数栈开销数据，默认模式为模式 0。
 - 模式 1：分析 elf 文件和源码地址，统计函数栈开销数据和函数所在源代码行号位置。

- 模式 2：分析反汇编文件和源码地址，统计函数栈开销数据和函数所在源代码行号位置。

----结束

7.2 栈分析

点击“Stack Analysis”按钮启动栈分析。分析完成后会在右侧展示分析结果。



页面内容说明

页面内容分为功能列表页面和调用图页面两个页面：

- 功能列表页面：以列表的形式展示所有函数的本地开销，可按本地开销大小进行升降排序。

功能列表

调用图

input search text

Q

函数名称	本地开销(Byte)	位置
hal_dmac_v100_interrupt_clear	16	
hal_dmac_v100_init	16	
hal_dmac_v100_deinit	16	
hal_dmac_v100_open	16	
hal_dmac_v100_close	16	
hal_dmac_v100_ch_enable	32	
hal_dmac_v100_get_block_ts	16	
hal_dmac_v100_ch_get_idle	16	
hal_dmac_v100_config_single_block	48	
hal_dmac_v100_add_lli_transfer	48	

<

1

2

3

4

5

...

430

>

10 / page

v

- 调用图页面：以树结构展示函数之间的调用层级关系，显示出函数在整个程序结构中的深度、最大开销、本地开销和源码位置。

功能列表

调用图

Input search text

Q

函数名称	深度	最大开销(Byte)	本地开销(Byte)	位置
g_trace_value	0	0	0	
hal_dmac_v100_interrupt_clear	0	32	16	
hal_dmac_v100_init	1	64	16	
hal_dmac_v100_deinit	1	64	16	
hal_dmac_v100_open	0	32	16	
hal_dmac_v100_close	0	32	16	
hal_dmac_v100_ch_enable	22	7328	32	
hal_dmac_v100_get_block_ts	0	32	16	
hal_dmac_v100_ch_get_idle	0	32	16	
hal_dmac_v100_config_single_block	3	192	48	

<

1

2

3

4

5

...

644

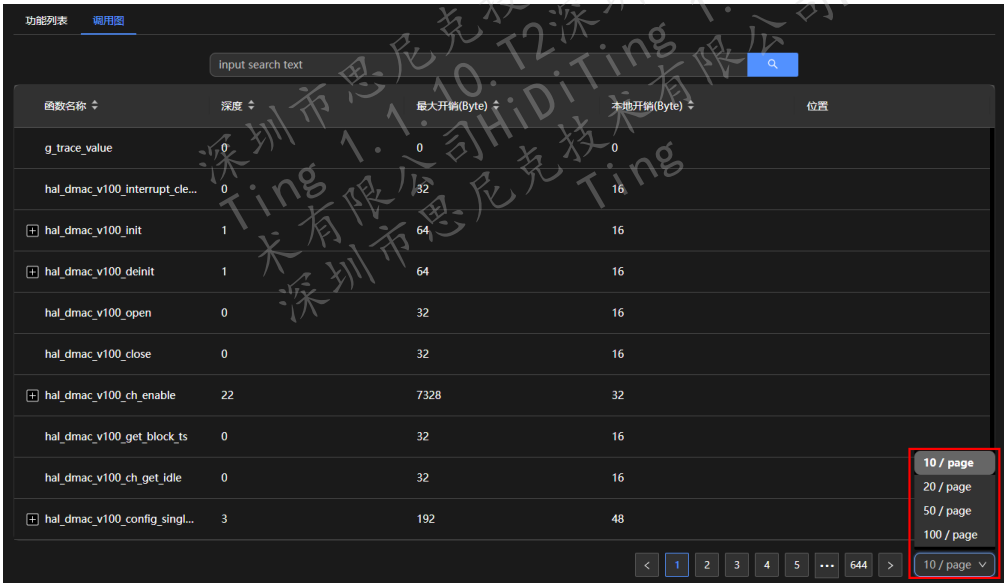
>

10 / page

辅助功能介绍：

- 搜索功能：在搜索框中可以进行函数查找，支持全词匹配和通配符模糊搜索。例如，关键字“os*init”可以搜索前面包含 os 字符后面包含 init 字符的所有函数；关键字“app_??_task”可以搜索“app_i2c_task”、“app_pwm_task”等“app_”字符和“_task”字符中间包含 3 个任意字符的所有函数。
- 排序功能：在可排序的数字列的表头，点击正/倒三角图标，切换排序规则，依次为默认排序、升序和降序。
 - 默认排序：全局初始状态，按照数据存放序列展示。
 - 升序：按照某一列升序排列。

- 降序：按照某一列降序排列。
- 设置每页显示记录条数：当记录过多，需要分页显示时，可调整每页记录条数。



统计项说明

- 最大开销：为当前函数所有子函数中最大的栈开销与循环调用的次数的乘积再加上自身的开销。
计算公式： $\max(\text{子函数 1 的自身栈开销}, \text{子函数 2 的自身栈开销}, \text{子函数 3 的自身栈开销}, \dots) \times \text{循环次数} + \text{函数的自身栈开销}$
- 本地开销：当前函数的自身栈开销。
- 深度：当前函数每增加一层子函数，深度增加 1 层。

7.3 镜像分析

点击 “Image Analysis” 按钮启动镜像分析。分析完成后会在右侧展示分析结果。

区域	起始地址	结束地址	容量	可用	已用	使用量(%)
BOOTUP_RAM	0x0000000000001000	0x0000101b4	436 B	48 B	388 B	88.99%
BOOTUP_LOAD	0x0000000007034000	0x703401b4	436 B	48 B	388 B	88.99%
FLASH_VERSION	0x000000000703401b4	0x7034020c	88 B	0 B	88 B	100.00%
FLASH_PROGRAM	0x0000000007034020c	0x706c0000	3583.49KB	947.70KB	2635.79KB	73.55%
PSRAM_APP_CODE	0x00000000c000000	0x6c100000	1024.00KB	1024.00KB	0 B	0.00%
PSRAM_APP_DATA	0x00000000c100000	0x6c200000	1024.00KB	1023.41KB	608 B	0.06%
PSRAM_HEAP	0x00000000c8c8b80	0x6cfc8b80	7168.00KB	7168.00KB	0 B	0.00%
L2M_GPU_PROGRAM	0x000000006024000	0x60280000	256.00KB	256.00KB	0 B	0.00%
L2M_DSP_PROGRAM	0x000000006010000	0x60140000	256.00KB	256.00KB	0 B	0.00%
ITCM	0x000000000000101b4	0x00040000	191.57KB	71.71KB	119.86KB	62.57%

分析结果分为内存区域、内存详细信息、文件大小、模块大小和文件夹大小 5 个页面。

- 内存区域页面：上半部分为各区域内存占比表格数据，下半部分显示当前选择区域的内存使用量占比。
- 内存详细信息页面：可查看详细的内存区域和内存分段的结构、运行地址、加载地址和容量大小。
- 文件大小页面：可查看文件结构和文件在各分段所占大小。
- 模块大小页面：相比文件大小页面，模块大小对链接库文件和目标文件进行归类，如 libgraphic_service.a、libgraphic_utils.a、libgraphic_ui.a 等都可归为 graphic 模块，方便查看 graphic 模块所有的文件所占空间大小。支持导出模块大小页面上的数据，导出格式为 excel 表格。在页面底部展示两个饼图，左边饼图展示所有模块的占比情况；右边饼图展示点选模块内部占比情况，点击列表中的一个模块节点后，右边饼图会展示这个模块的内部占比。

内存区域

内存详细信息

文件大小

模块大小

文件夹大小

导出excel

input search text

模块名称	总量	.text	.data	.bss	startup	.version_str	.psram_data	.psram_bss	.plt_ramtext	.sysintcall.ini
dfx	46599	44926	122	2867	0	0	0	0	1551	0
kernel-base	16370	0	16	540	0	0	0	0	16354	28
libriscv.a	2066	0	12	24	0	0	0	0	2054	4
update	19267	11426	152	568	0	0	0	0	7689	0
media	465586	460817	4733	3744	0	0	0	0	36	0
graphic	680512	678671	1821	9979	0	0	0	0	20	0
bgh	258105	254558	3543	2065	0	0	0	0	4	0
kernel-init	571	571	0	26536	0	0	0	0	0	16
targets	880	400	480	16	0	0	0	0	0	8
driver-base	80	80	0	24	0	0	0	0	0	4
compat-linux	1209	1209	0	8	0	0	0	0	0	8

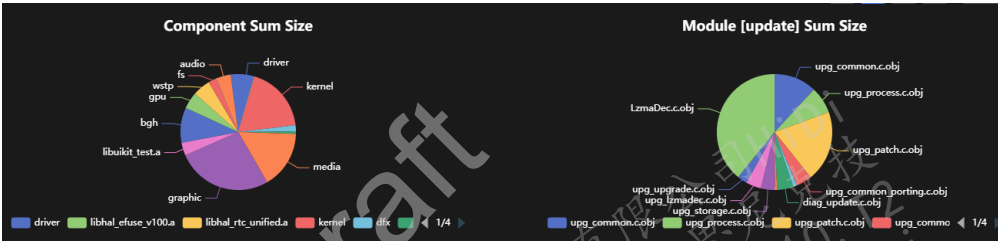
<

1

2

>

15 / page



- 文件夹大小页面：将文件大小按目录结构展示，可导出为 Excel 表格。按目录层级形式显示文件和文件夹大小，单个子目录的路径会有层级压缩。

内存区域

内存详细信息

文件大小

模块大小

文件类型

导出excel

input search text

Q

文件名称	总量	.text	.data	.bss	startup	version_str	.psram_data	.psram_bss	.plt_ramtext	.sysintcall.ini
drivers\chips\brandy	23095	6763	457	5501	0	0	0	0	15875	0
kernel	11450	2893	32	36	0	0	0	0	8525	0
lib	398280	361860	4219	56279	0	0	0	0	32201	68
middleware\chips\brandy	10370	4160	127	1025	0	0	0	0	6083	0
uikit	370513	369340	1145	6990	0	0	0	0	28	0
interim_binary\brandy\libs	759706	749868	9834	8326	0	0	0	0	4	0
vendor\segger\triscv70\libsegger.a	11172	10532	640	0	0	0	0	0	0	0
services\gui\hal	17296	16968	328	2736	0	0	0	0	0	0
open_source\7-zip-lzma-sdk\liblzma.a	6688	6688	0	0	0	0	0	0	0	0
freetype\libfreetype_static.a	101217	101217	0	0	0	0	0	0	0	0
qrcodegen\libqrcodegen.a	18076	18072	4	0	0	0	0	0	0	0

<

1

2

>

15 / page

辅助功能介绍：

- 搜索功能：在首列（函数名称）中比对搜索输入框中输入的关键字，列出包含关键字的函数。支持全词匹配和通配符模糊搜索。
例如，关键字 `os*init` 可以搜索前面包含 `os` 字符后面包含 `init` 字符的所有函数，`liteos_wifi_init`、`os_app_init`、`hi_os_led_init` 等函数都能被搜索到；关键字 `"app_???_task"` 可以搜索 `"app_i2c_task"`、`"app_pwm_task"` 等 `"app_"` 字符和 `"_task"` 字符中间包含 3 个任意字符的所有函数。
- 排序功能：在可排序的数字列的表头，点击正/倒三角图标，切换排序规则，依次为默认排序、升序和降序。
- 总量筛选：总量列的右边有总量的统计项筛选功能，可灵活调节各列是否归入总量统计。

内存区域 内存详细信息 文件大小 模块大小 文件类大小

input search text

文件名称	总量	.text	.data	.bss	startup	.version_str	.psram_data	.psram_bss	.plt_ramtext	.sysintcall.ini
libbrandy_app_standard.a	4754	0	642	114	388	0	0	0	4754	0
libbuild_version.a	28	0	0	0	0	84	0	0	0	0
libapp_at_service.a	18490	208	0	0	0	0	13	535	0	0
libaudio_at_service.a	1659	0	0	0	0	0	0	4	0	0
libbt_manager_at_service.a	25188	0	0	0	0	0	0	14	0	0
libgraphic_at_service.a	3494	3494	0	0	0	0	0	15	0	0
libmedia_at_service.a	9059	9059	0	0	0	0	0	14	0	0
libhal_cpu_core.a	18	0	0	0	0	0	0	0	18	0
libhal_cpu_trace.a	824	0	0	0	0	0	0	0	824	0
libhal_dma_v100.a	2957	0	48	252	0	0	0	0	2909	0
libhal_gpio_v100.a	2594	0	32	1156	0	0	0	0	2562	0

1 2 3 4 5 ... 13 > 15 / page

8

添加文件到编译框架

插件功能介绍：该插件支持开发者一键生成编译构建模板，同时支持自动添加文件到编译模板。

插件使用注意事项：

- 目前仅支持 Deveco 导入的工程项目，且编译框架目前仅支持 CMake。
- 要使用自动添加文件功能，需要先将目标文件夹的路径加入插件的“Cmake-watch”配置项中，可以通过选中目录右键添加，或者将路径复制加入插件配置中，具体修改方式请参见“8.4 自动添加文件到编译框架”。
- 文件格式识别目前仅支持.c、.cpp、.s 文件夹。
- 本插件仅支持 CMake 模板构建，不会变更整体的编译架构。因此将本插件的 CMake 模板覆写原本的 CMake 文本时，请谨慎操作，否则会导致工程编译失败。

8.1 生成 CMake 文件模板

8.2 插入 CMake 文件模板

8.3 插入 CMake 日志

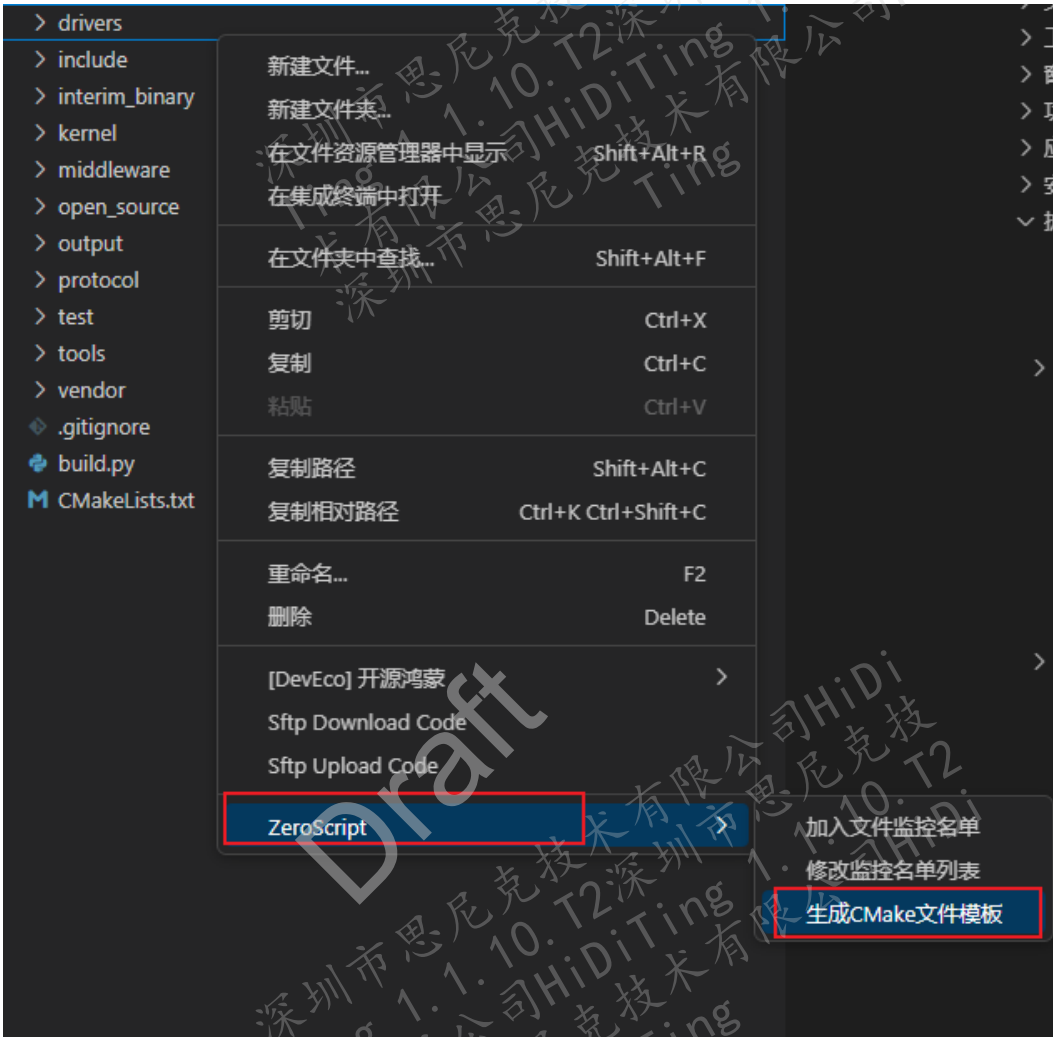
8.4 自动添加文件到编译框架

8.1 生成 CMake 文件模板

功能说明：在选择目录下创建一个 CMakeLists 模板文件，同时扫描目录下的文件夹及部分格式文件，将其信息添加到生成的“CMakeLists.txt”文件中。

生成 CMake 文件模板步骤如下：

步骤 1 选择要生成模板的文件夹，右键选择“ZeroScript”，点击“生成 CMake 文件模板”按钮。



步骤 2 生成模板。

```
M CMakeLists.txt X
test_plugin > M CMakeLists.txt
1  #=====
2  # @brief   cmake file
3  # Copyright (c) CompanyNameMagicTag 2022-2023. All rights reserved.
4  #=====
5  set(COMPONENT_NAME "test_plugin")
6
7  set(SOURCES
8      ${CMAKE_CURRENT_SOURCE_DIR}/aaa.c
9      ${CMAKE_CURRENT_SOURCE_DIR}/123.c
10 )
11
12 set(PUBLIC_HEADER
13 )
14
15 set(PRIVATE_HEADER
16 )
17
18 set(PRIVATE_DEFINES
19 )
20
21 set(PUBLIC_DEFINES
22 )
23
24 # use this when you want to add ccflags like -include xxx
25 set(COMPONENT_PUBLIC_CCFLAGS
26 )
27
28 set(COMPONENT_CCFLAGS
29 )
30
31 set(WHOLE_LINK
32     true
33 )
34
35 set(MAIN_COMPONENT
36     false
37 )
38
39 build_component()
40
```

步骤 3 修改后编译成功。

```
[1879048192, 1879371776, 1882456064, 1879638816, 1880227840, 1881161728, 1882408128]
[131072, 4096, 3670016, 589824, 933888, 1310720, 16384]
[69224, 1024, 2594816, 435696, 327288, 1030220, 16384]
[0, 100, 18, 20, 22, 26, 100]
packet success!

please check the compilation log: C:\Users\...\.dev\code\ice-tool\logs\build\build.log
===== [SUCCESS] Took 338.77 seconds =====

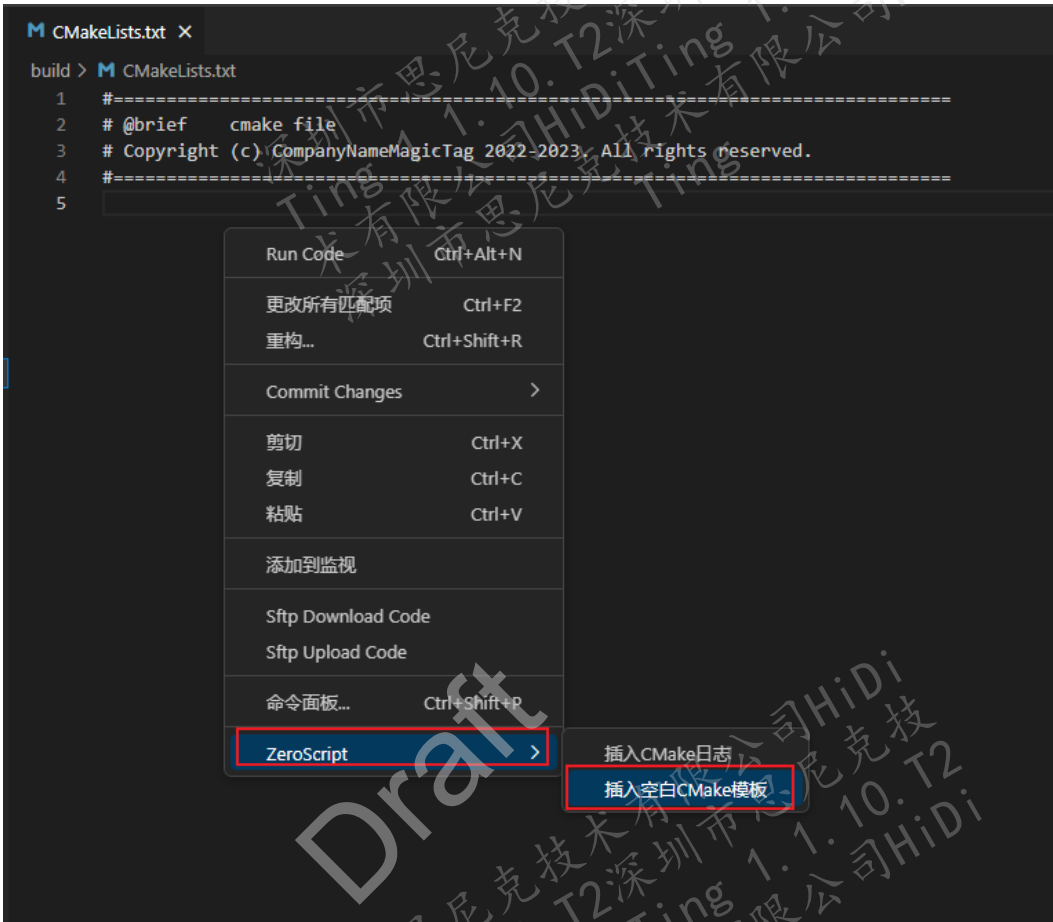
[ ] 终端将被任务重用，按任意键关闭。
```

----结束

8.2 插入 CMake 文件模板

功能说明：在 “CMakeLists.txt” 中插入预定义的模板文本，不包含文件夹下的任何文件信息。

步骤 1 打开需要插入模板文件的“CMakeLists.txt”文件，光标选中需要插入文本的所在行，右键选择“ZeroScript”，点击“插入空白 CMake 模板”按钮。



步骤 2 文本模板插入。

```
1 #=====
2 # @brief   cmake file
3 # Copyright (c) CompanyNameMagicTag 2022-2023. All rights reserved.
4 #=====
5
6 set(COMPONENT_NAME "build")
7
8 set(SOURCES
9 )
10
11 set(PUBLIC_HEADER
12 )
13
14 set(PRIVATE_HEADER
15 )
16
17 set(PRIVATE_DEFINES
18 )
19
20 set(PUBLIC_DEFINES
21 )
22
23 # use this when you want to add ccflags like -include xxx
24 set(COMPONENT_PUBLIC_CCFLAGS
25 )
26
27 set(COMPONENT_CCFLAGS
28 )
29
30 set(WHOLE_LINK
31     true
32 )
33
34 set(MAIN_COMPONENT
35     false
36 )
37
38 build_component()
39
```

步骤 3 修改后编译成功。

```
[1879048192, 1879371776, 1882456064, 1879638816, 1880227840, 1881101728, 1887408128]
[131072, 4896, 3670816, 589824, 933888, 1310720, 16384]
[69224, 1024, 2594816, 435696, 327288, 1030220, 16384]
[0, 100, 18, 20, 22, 26, 100]
packet success!

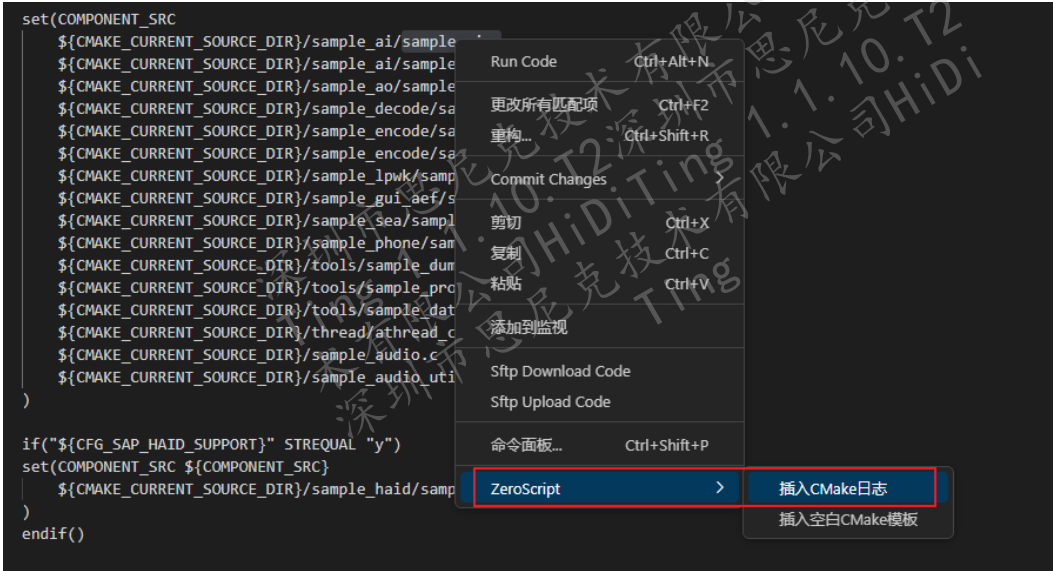
please check the compilation log: C:\Users\...\.devext-device-tool\logs\build\build1.log
[SUCCESS] Took 338.77 seconds
```

----结束

8.3 插入 CMake 日志

功能说明：在 “CMakeLists.txt” 中将选中的文本生成日志信息插入。

步骤 1 打开需要插入日志的 “CMakeLists.txt” 文件，光标选中需要插入日志的文本部分，右键选择 “ZeroScript” ，点击 “插入 CMake 日志” 按钮。



步骤 2 插入日志。

```
`${CMAKE_CURRENT_SOURCE_DIR}/sample_ai/sample_ai.c`  
MESSAGE(STATUS "sample_ai.c: `${sample_ai.c}`")
```

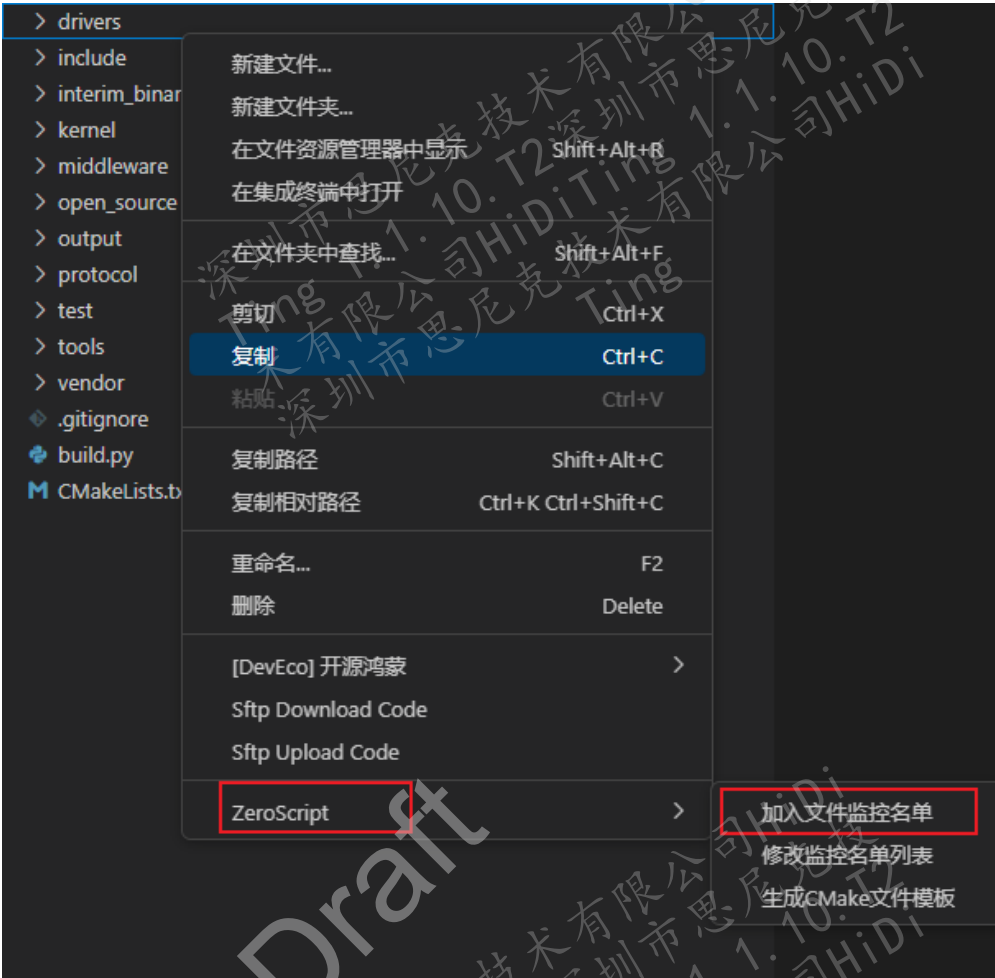
----结束

8.4 自动添加文件到编译框架

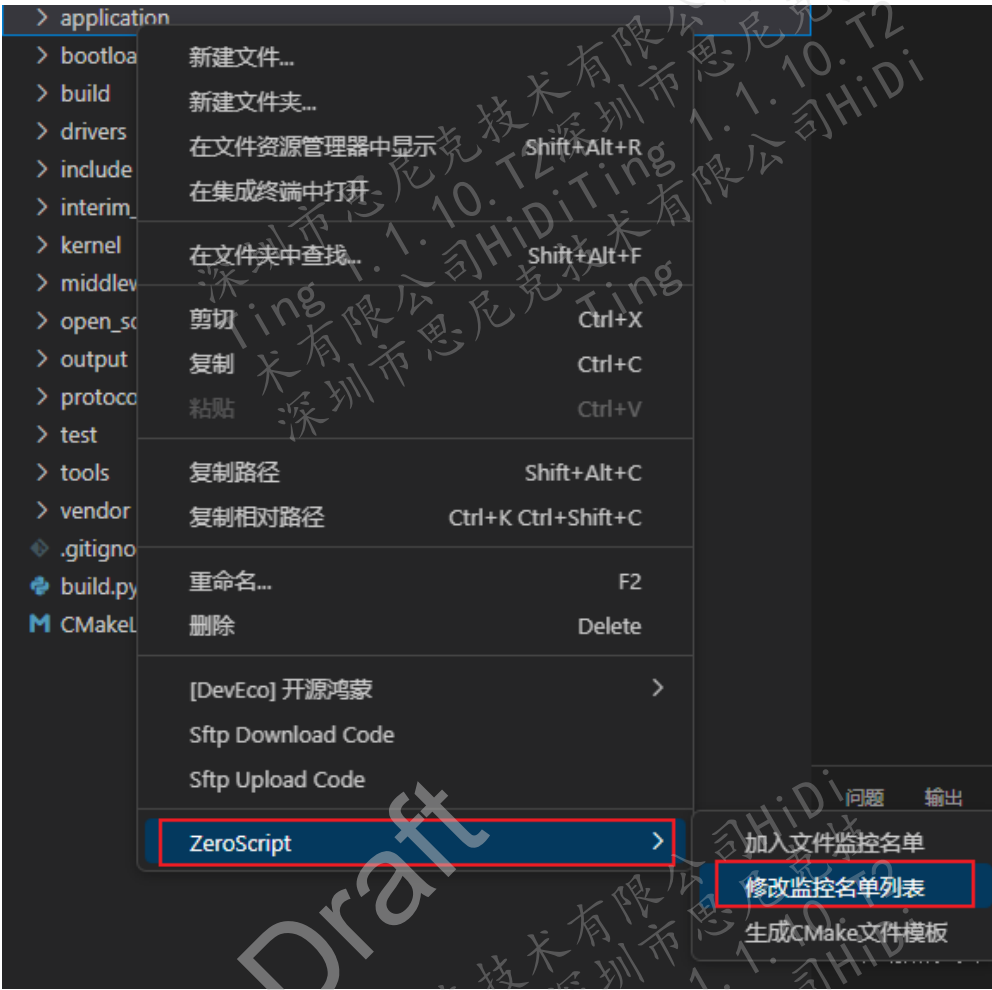
- 功能说明：自动添加.c、.cpp、.s 或者文件夹到 “CMakeLists.txt” 编译框架中。
- 功能依赖：
- 文件夹路径需要在文件识别的白名单中。
 - 文件中需要存在 “CMakeLists.txt” 文件。
 - “CMakeLists.txt” 中需要存在至少一个 Component 组件，才能支持文件自适应加入编译框架。
 - VsCode 打开的工程必须为 Deveco 项目，且包含导入生成的 “.deveco/deveco.ini”，其中的 framework 为 CMake。

步骤 1 将需要支持自适应编译的文件夹路径（建议使用相对路径）添加到插件配置项中，监控路径名单增删方式如下。

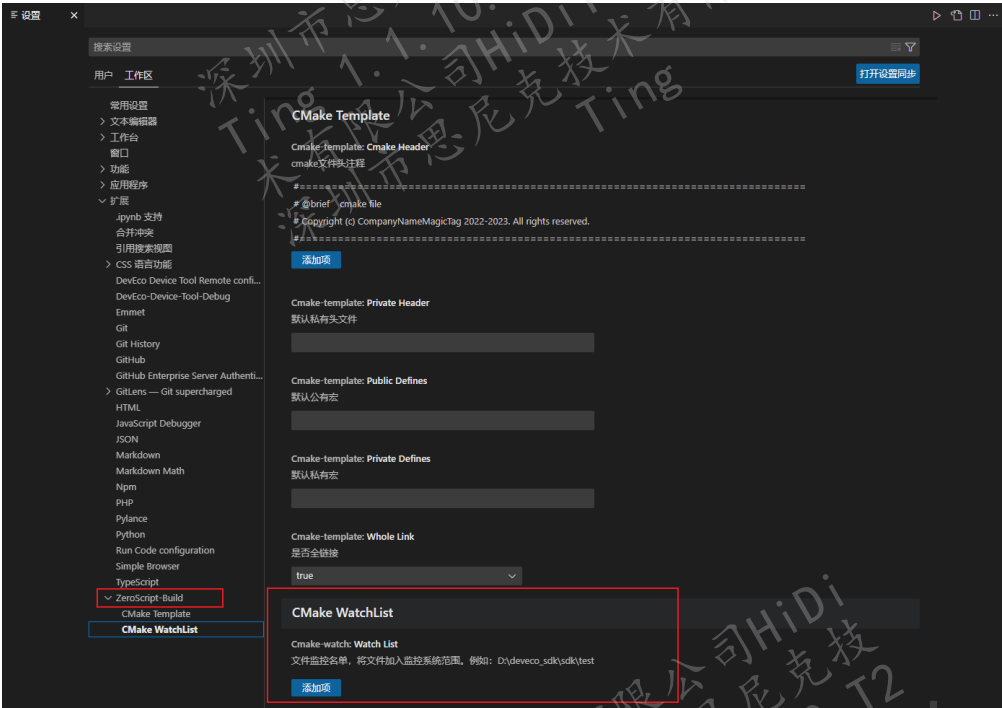
1. 右键加入文件监控名单。



右键修改监控名单列表。

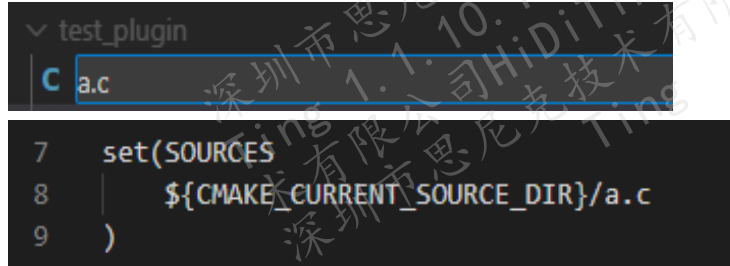


2. VsCode 设置插件配置属性

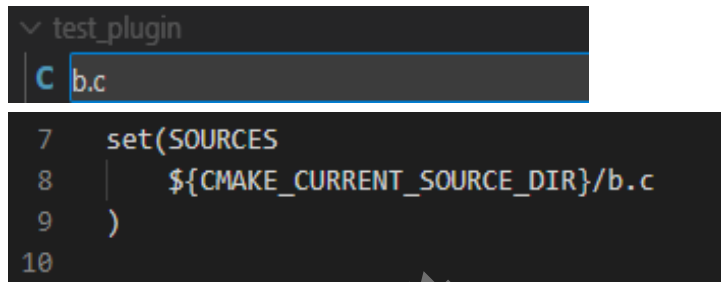


步骤 2 在该文件夹中新增/重命名/删除 .c、.cpp、.s 或者文件夹, “CMakeLists.txt” 将随着文件变更自动修改相关文本内容。

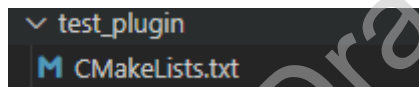
1. 新增文件。



2. 重命名文件。

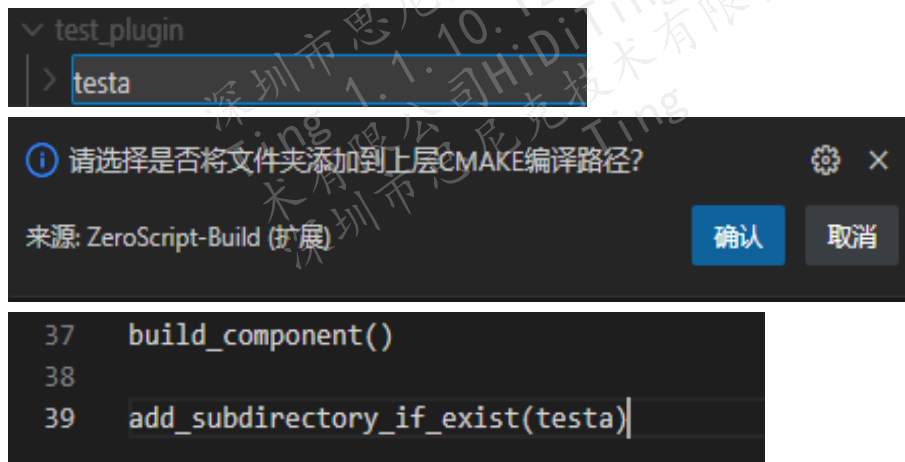


3. 删除文件。



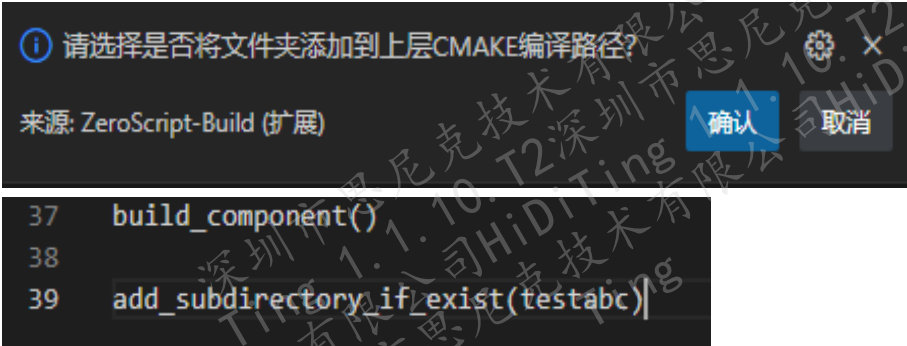
```
CMakeLists.txt X
test_plugin > M CMakeLists.txt
1  #=====
2  # @brief   cmake file
3  # Copyright (c) CompanyNameMagicTag 2022-2023. All rights reserved.
4  #=====
5  set(COMPONENT_NAME "test_plugin")
6
7  set(SOURCES
8  )
9
10 set(PUBLIC_HEADER
11 )
12
13 set(PRIVATE_HEADER
14 )
15
16 set(PRIVATE_DEFINES
17 )
18
19 set(PUBLIC_DEFINES
20 )
21
22 # use this when you want to add ccflags like -include xxx
23 set(COMPONENT_PUBLIC_CCFLAGS
24 )
25
26 set(COMPONENT_CCFLAGS
27 )
28
29 set(WHOLE_LINK
30 | true
31 )
32
33 set(MAIN_COMPONENT
34 | false
35 )
36
37 build_component()
38
```

4. 新增文件夹，需要点击确认才能加入到编译框架。



5. 重命名文件夹，需要点击确认才能加入到编译框架。

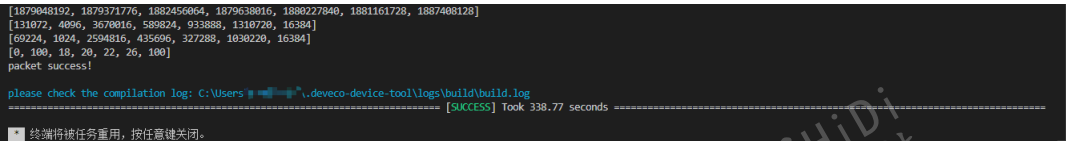




6. 删除文件夹。



步骤 3 修改后编译成功。



----结束

9 工程调试

📖 说明

单板的版本与选择的 elf 版本应一致，并且单板处于唤醒状态，以保证调试功能正常运行。

9.1 调试工具配置

9.2 调试配置选项

9.3 启动调试

9.4 常用调试功能

9.5 调试实例介绍

9.1 调试工具配置

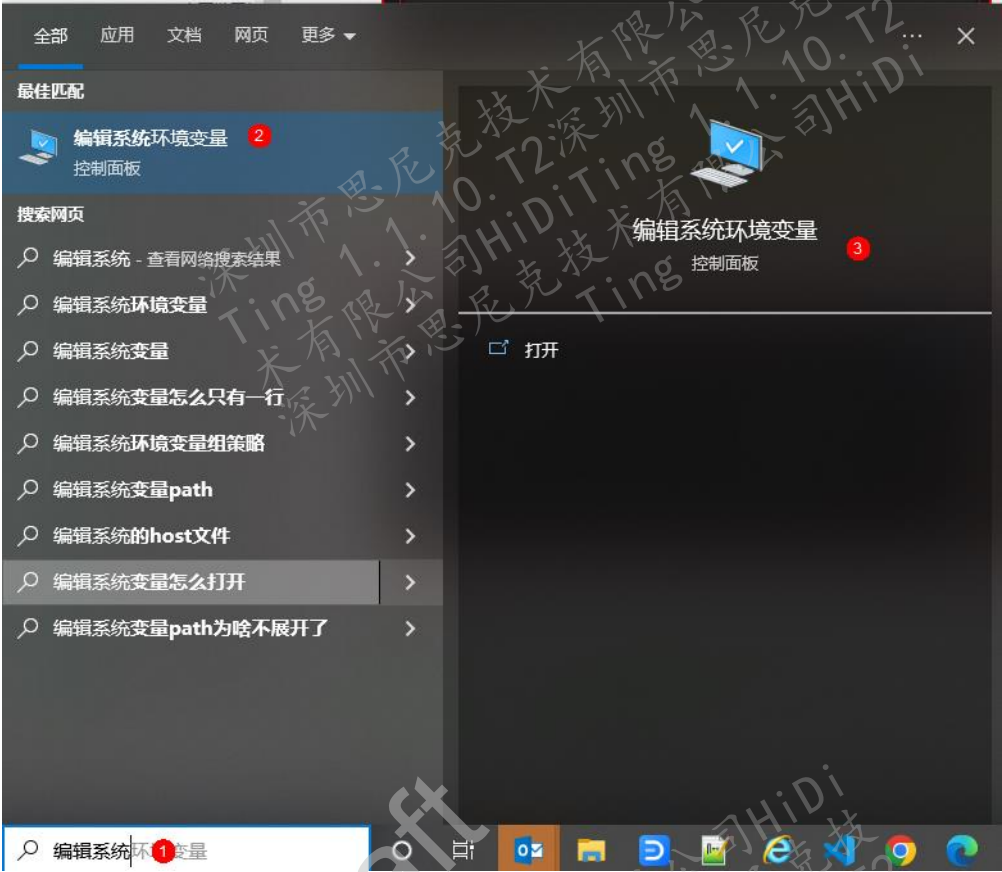
当前 DevEco Device Tool 工具支持 BrandyK100 单板调试功能，该功能依赖于 BrandyK100 交付件中的 GDB，GDB 路径配置当前依赖 “compiler_bin_path” 配置，如已配置，可忽略。

JLINK 环境依赖：

📖 说明

J-Link 推荐使用 V6.98c 版本的驱动。

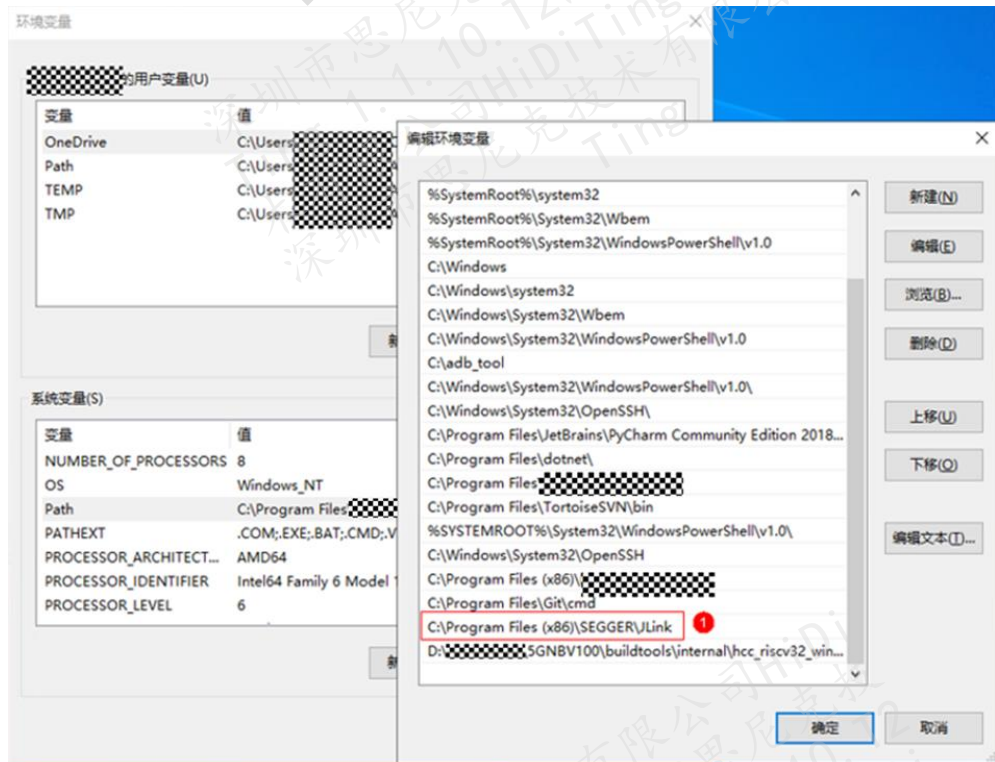
1. 环境变量配置：JLINK 软件的路径也需要配置在系统环境变量 Path 中，在输入框中输入【编辑系统环境变量】，按下图步骤打开。



选择“环境变量”→“系统变量”，找到“path”变量。



如下图所示。配置完 JLINK 环境变量需要重启 Device Tool 工具，保证路径生效。



2. 硬件仿真器要求：推荐 JLINK 使用 V11 及以上版本。



9.2 调试配置选项

步骤 1 选择要调试的工程：在 IDE 主界面中，选择要调试的工程，打开“工程配置”。

步骤 2 修改调试选项，当前已给用户默认配置，用户仅需修改 debug_elf 设置，指定到编译生成的 elf 文件路径。

----结束

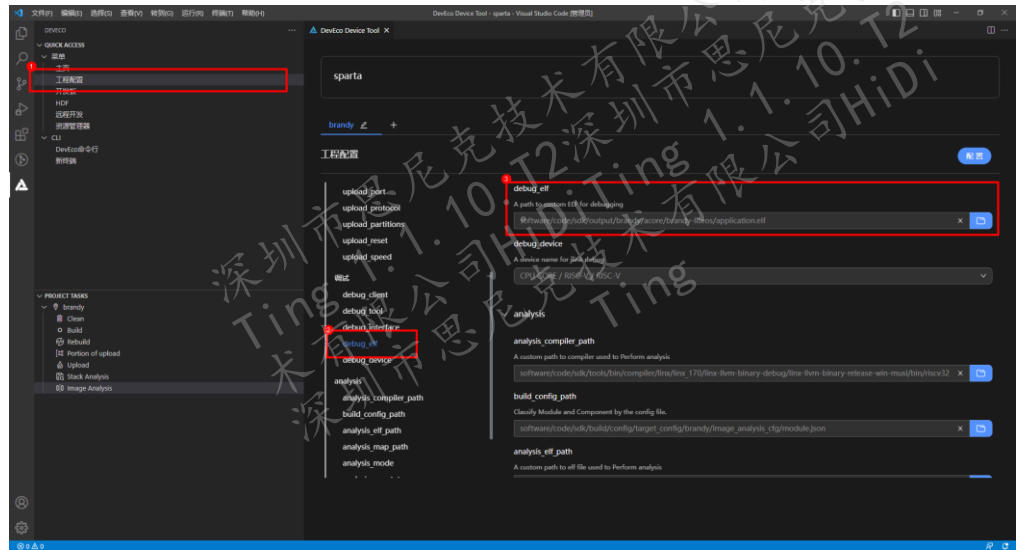


如图所示，依次配置为：

- [debug_client]：调试客户端，默认 gdb。
- [debug_tool]：调试工具，默认 jlink。
- [debug_interface]：选择调试器的连接模式 jtag 或 swd，默认配置成 swd。
- [debug_elf]：配置生成的 elf 文件路径。
- [debug_device]：设备名称，CPU CORE/RISC-V/RISC-V。

说明

当前除【debug_elf】，其他配置已经自动配置完成，【debug_elf】配置方法如下：



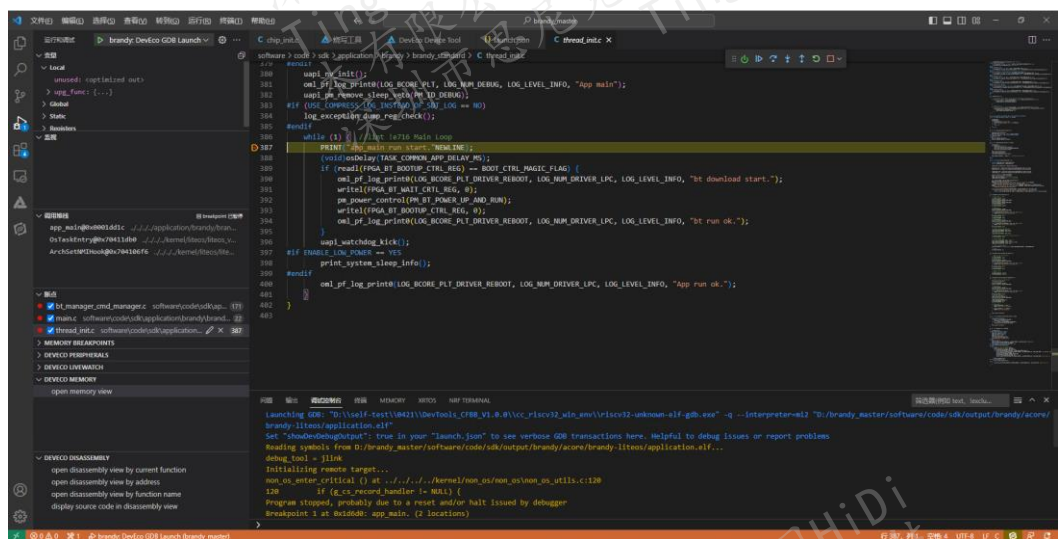
9.3 启动调试

步骤 1 单击 IDE 左侧栏 “” 进入调试界面。

- Launch 模式：系统重启，暂停 CPU，设置 PC 指针从头开始运行程序。
- Attach 模式：程序正在运行中，暂停 CPU，程序直接停在 CPU Halt 处。

步骤 2 单击运行和调试右侧的  启动调试。

步骤 3 启动调试成功。调试成功示例如下图，出现下面图标说明已经进入调试。



说明

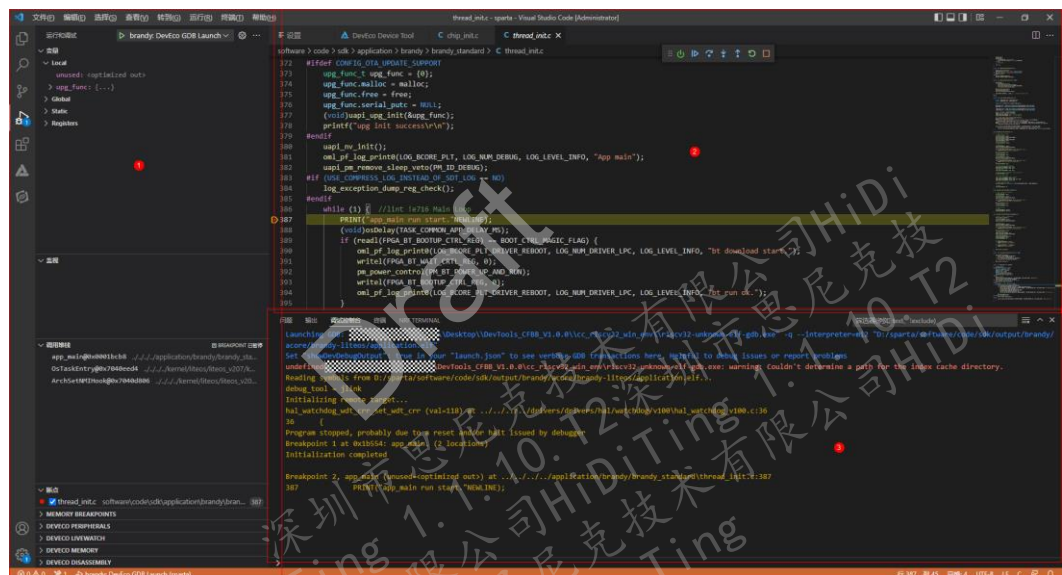
当前仅支持一个 Launch 进行调试，不支持启动多个 Launch 调试。

结束

9.4 常用调试功能

调试工作界面如下图所示，主要由以下 3 个部分组成：

- ① 调试侧边栏
- ② 调试功能区
- ③ 调试控制台



调试侧边栏

调试侧边栏集合了调试常用功能，包括变量、监视、调用堆栈、断点、查看外设寄存器、查看内存地址数据、查看反汇编。

调试功能区

启动调试功能后，当代码执行到设置的断点时，程序会暂停，可以根据调试功能区的按钮进行代码的调试。



- : Reset device, 复位设备。

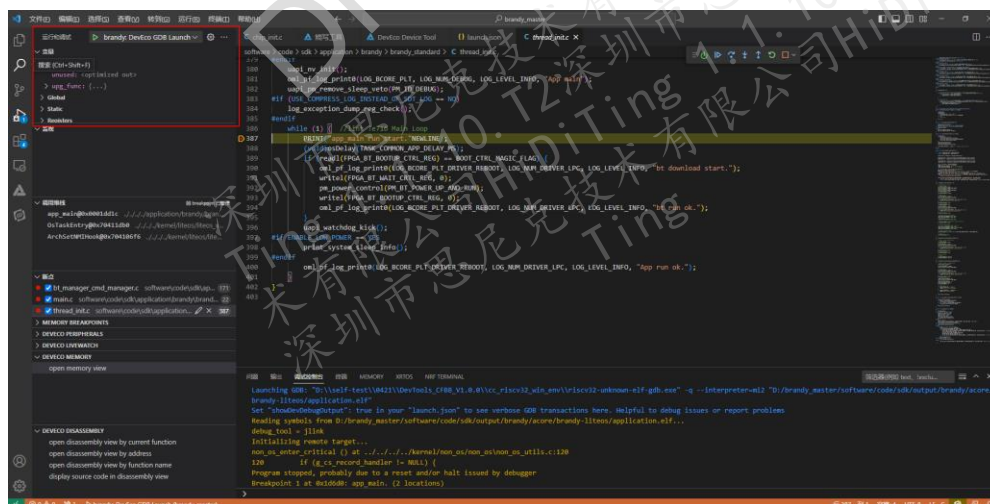
- : Continue ("F5"), 当程序执行到断点时停止执行, 点击此按钮程序继续执行。
- : Step Over ("F10"), 在单步调试时, 直接前进到下一行 (如果在函数中存在于子函数时, 不会进入子函数内单步执行, 而是将整个子函数当作一步执行)。
- : Step Into ("F11"), 在单步调试时, 遇到子函数后, 进入子函数并继续单步执行。
- : Step Out ("Shift+F11"), 在单步调试执行到子函数内时, 点击 Step Out 会执行完子函数剩余部分, 并跳出返回到上一层函数。
- : Restart ("Ctrl+Shift+F5"), 重新启动调试。
- : Stop ("Shift+F5"), 停止调试任务, 断开连接。

调试控制台

调试控制台用来输出调试时的打印信息，也可以输入命令与调试器交互。

- 变量查看

当运行到断点处暂停时，可以在变量界面查看变量值的变化。



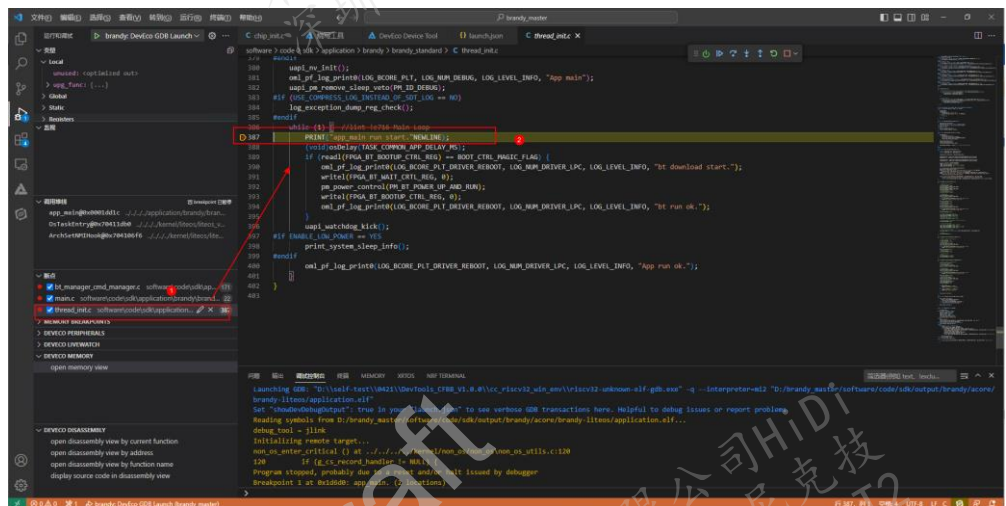
支持如下 4 种变量类型:

- 局部变量
- 全局变量（可能会被编译器优化，可以使用关键字 `volatile` 来规避这个问题。）
- 静态变量（可能会被编译器优化，可以使用关键字 `volatile` 来规避这个问题。）
- risc-v 寄存器

管理函数断点

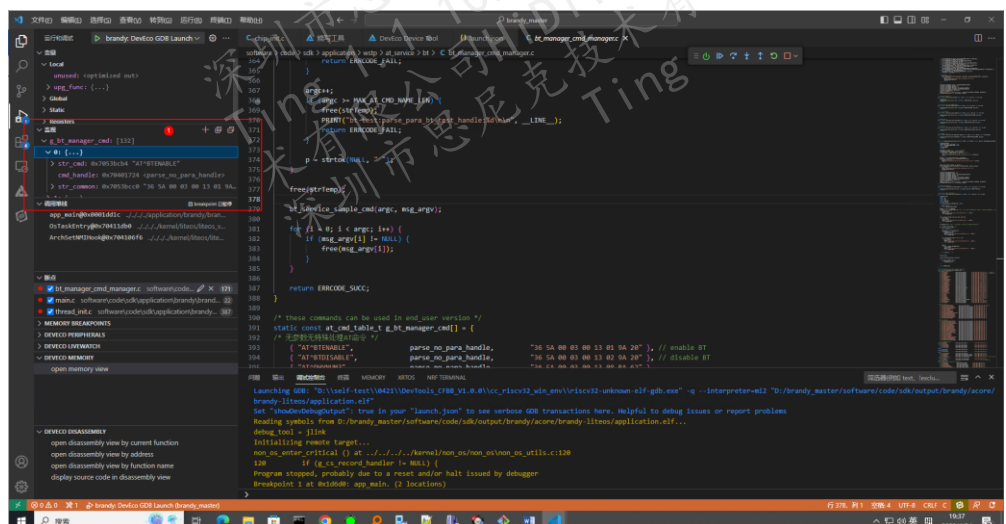
插入函数断点的方式有多种方法：

- 用侧边栏的“断点”功能手动添加断点函数，点击左侧断点一栏右上角的 + 号，在文本框中输入需要断点的函数。
- 用鼠标点击代码编辑区代码行最左侧的位置，位置参考图片中红色圆点位置，点击后出现红色圆点表示添加断点成功。
- 使用快捷键 “F9”。可以在断点菜单下使能、去使能、添加和删除断点。



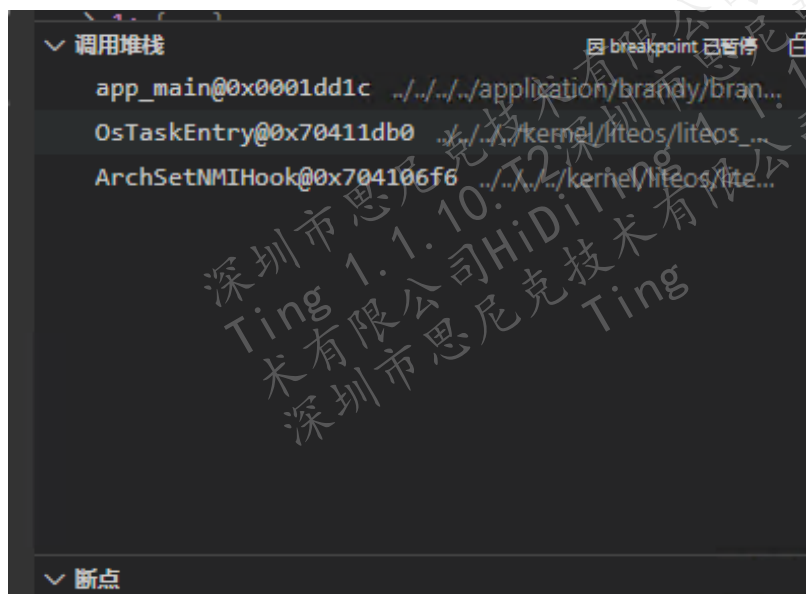
监视功能

在调试过程中，可以通过“监视”查看变量（包括局部变量、全局变量以及静态变量）的取值来判断程序的计算结果是否有误，从而快速进行代码检查。



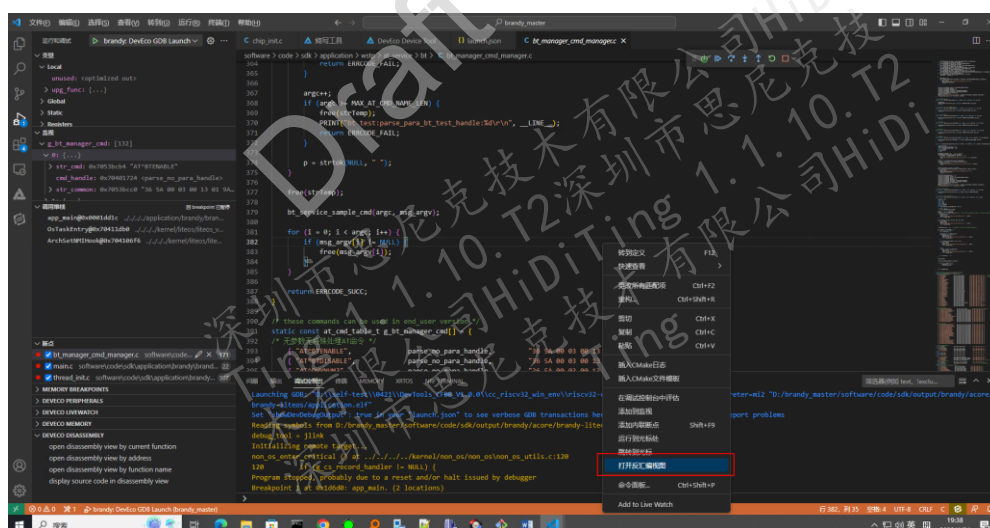
查看调用栈

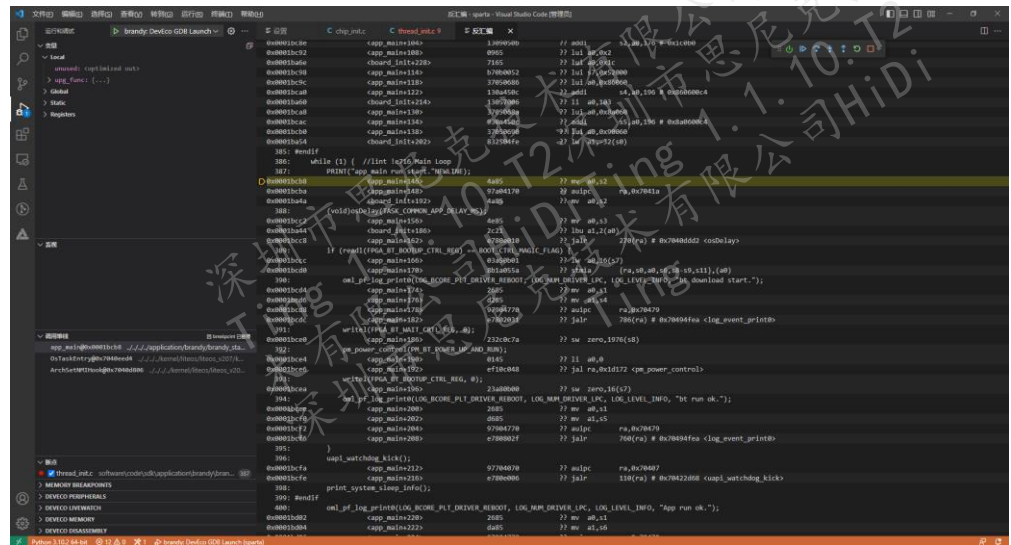
在调试过程中，可以通过查看调用栈来分析主程序调用的各子程序的调用关系，从而快速进行代码检查，如下图所示。



- 查看反汇编

在调试过程中，通过查看反汇编，可以确认代码的汇编实现，从而快速进行代码检查。通过鼠标在代码界面中点击右键，选中“打开反汇编视图”功能就可以进入代码的反汇编视图，如图所示。



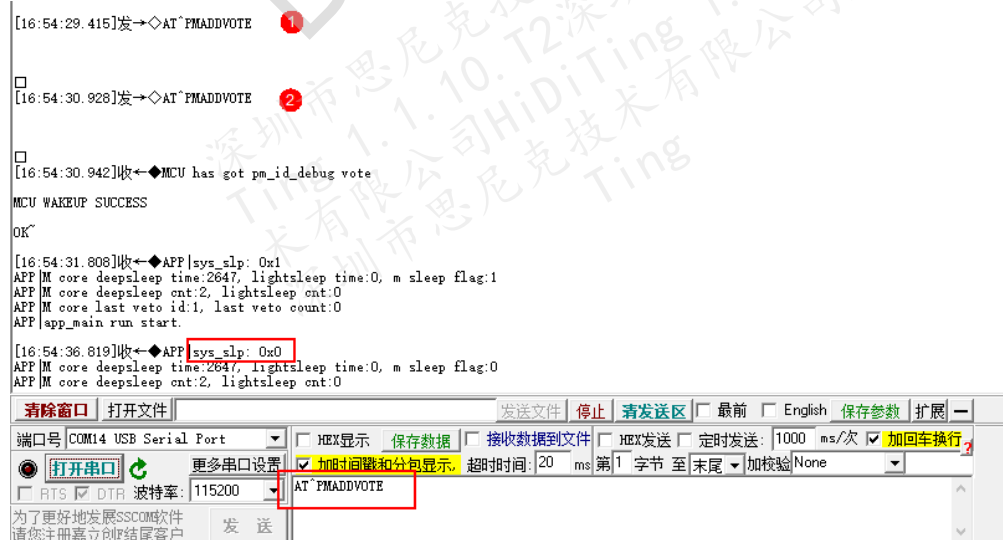


9.5 调试实例介绍

说明

调试单板前需要保证当前单板运行的程序，和调试时选择的 elf 文件是一致的，即同一时刻编译产生的文件。

1. 当前程序定期会进入深睡模式，会关闭调试寄存器，因此调试前，需要退出深睡模式，在串口工具下，输入两次 “AT^PMADDDVOTE”，如下图所示：



2. 由于业务进程普遍存在心跳喂狗，直接进行 “gdb attach” 会触发进程挂起，首先需要关闭看门狗。

- a. 代码中需要注释开狗函数，在

“application\brandy\brandy_standard\kernel_init.c” 文件中的 kernel_init 函

数中注释下图红框显示函数 chip_watchdog_init(), 重新编译, 然后烧录镜像到调试单板上。如下图所示。

```
C kernel_init.c 7 x
software > code > sdk > application > brandy > brandy_standard > C kernel_init.c > kernel_init(void)
166 (void)ArchICacheEnable(CACHE_32KB);
167 (void)ArchICachePrefetchEnable(CACHE_PREF_1_LINES);
168
169 (void)ArchDCacheEnable(CACHE_32KB);
170 (void)ArchDCachePrefetchEnable(CACHE_PREF_1_LINES, CACHE_PREF_1_LINES);
171 }
172 #endif
173
174 static void chip_watchdog_init(void)
175 {
176     watchdog_turnon_clk();
177     watchdog_func_adapt(CHIP_WDT_TIMEOUT_32S);
178     uapi_watchdog_init(CHIP_WDT_TIMEOUT_32S);
179     uapi_watchdog_enable(WDT_MODE_INTERRUPT);
180 }
181
182 void kernel_init(void)
183 {
184     // chip_watchdog_init();
185
186     pmu1_dsp_l2ram_config(DSP_L2RAM_AREA_SIZE);
187     copy_from_flash_to_l2ram();
188
189     #ifdef CONFIG_PSRAM_SUPPORT
190     copy_from_flash_to_psrn();
191     #endif
192
193     panic_init();
194
195     #if defined(__FREERTOS__)
196     /* os kernel adapt config */
197     vPortHwiInit();
198
199     /* enable trace and debug */
```

3. 在“application\brandy\brandy_standard\thread_init.c”文件的 app_main 的红框指定位置打下断点（如图），然后启动调试，app_main 中循环执行，可以在此处断点执行，表示已经进入调试页面。

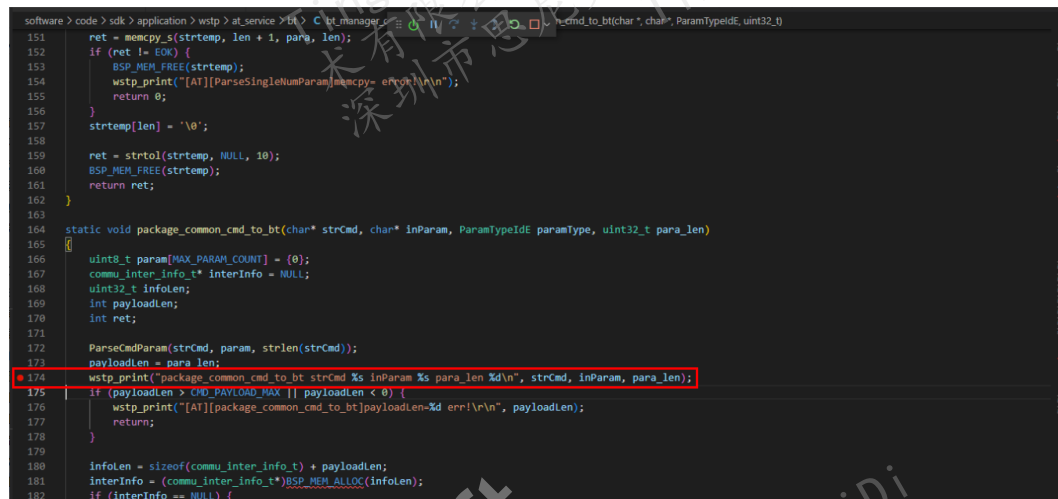
```
DevEco Device Tool 烧写工具 config.py target.config.py common.config.py C thread_init.c 6 x
software > code > sdk > application > brandy > brandy_standard > C thread_init.c > ...
270 unused(unused);
271 #ifdef CONFIG_OTA_UPDATE_SUPPORT
272     upg_check_upgrade();
273 #endif
274     dfx_set_log_enable();
275     /* nv升级必须在文件系统初始化之后 */
276 #ifdef CONFIG_OTA_UPDATE_SUPPORT
277     (void)kv_upg_upgrade_task_process();
278 #endif
279
280 #ifdef SUPPORT_MEDIA_AUDIO_MANAGER
281     bool success = AudioManagerInit();
282     PRINT("audio manager init %s\r\n", success ? "ok" : "error");
283 #endif
284     oml_pf_log_print0(LOG_BCORE_PLT, LOG_NUM_DEBUG, LOG_LEVEL_INFO, "App main");
285     #if (USE_COMPRESS_LOG_INSTEAD_OF_SDT_LOG == NO)
286     log_exception_dump_reg_check();
287     #endif
288     uapi_pm_request_mode(PM_ID_DEBUG, PM_MODE_LP);
289     PRINT("app_main run start.\n");
290     while (1) { //lint !e716 Main Loop
291         (void)osDelay(g_app_delay_ms);
292         uapi_watchdog_kick();
293     #ifdef XTS_SUPPORT
294         PRINT("app_main run start.\n");
295         print_system_sleep_info();
296         oml_pf_log_print0(LOG_BCORE_PLT_DRIVER_REBOOT, LOG_NUM_DRIVER_LPC, LOG_LEVEL_INFO, "App run ok");
297     #endif
298     }
299 }
300
```

说明

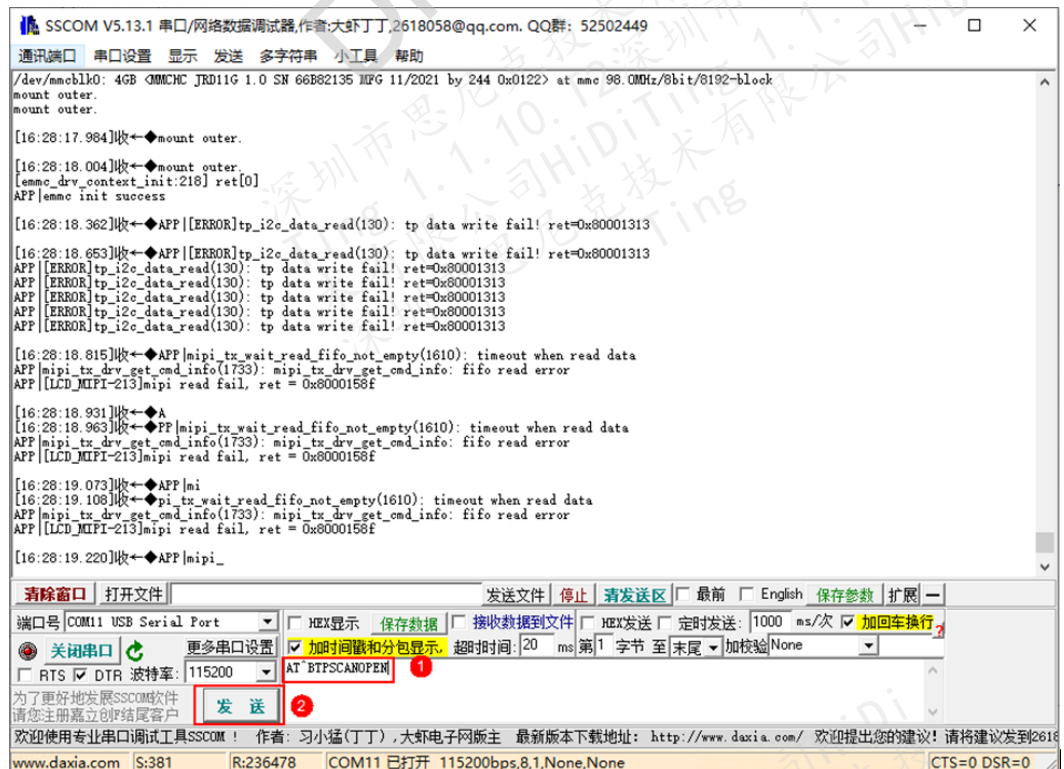
停留在 app_main 以后，需要取消该断点，否则一直在此处循环。

用户态代码如何进一步调试，下面以调试 AT 命令举例：

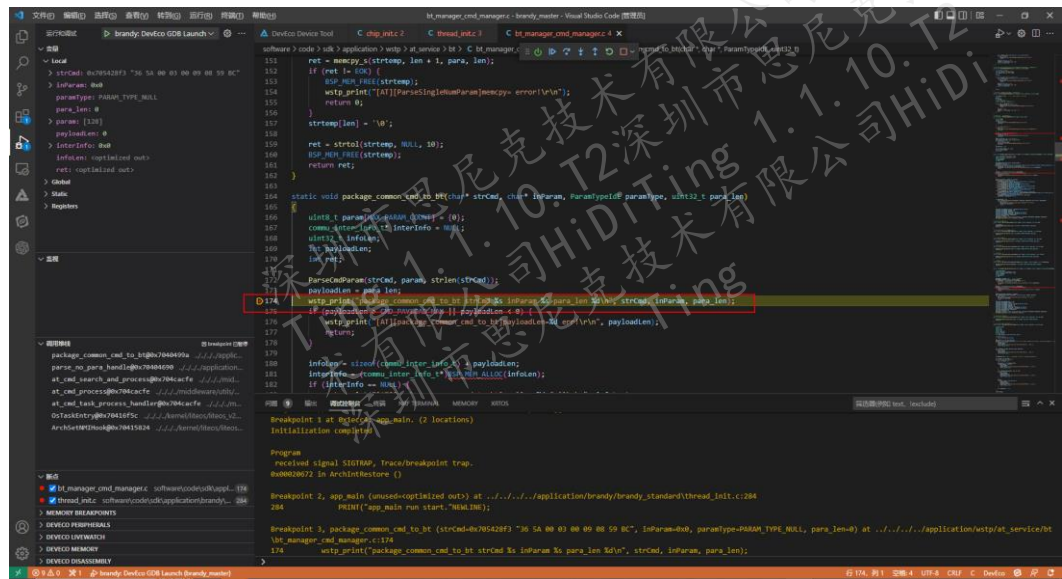
步骤 1 在 “application\wstpl\at_service\bt\bt_manager_cmd_manager.c” 文件的 package_common_cmd_to_bt 函数如下图红框显示处中打下断点，如下图所示。



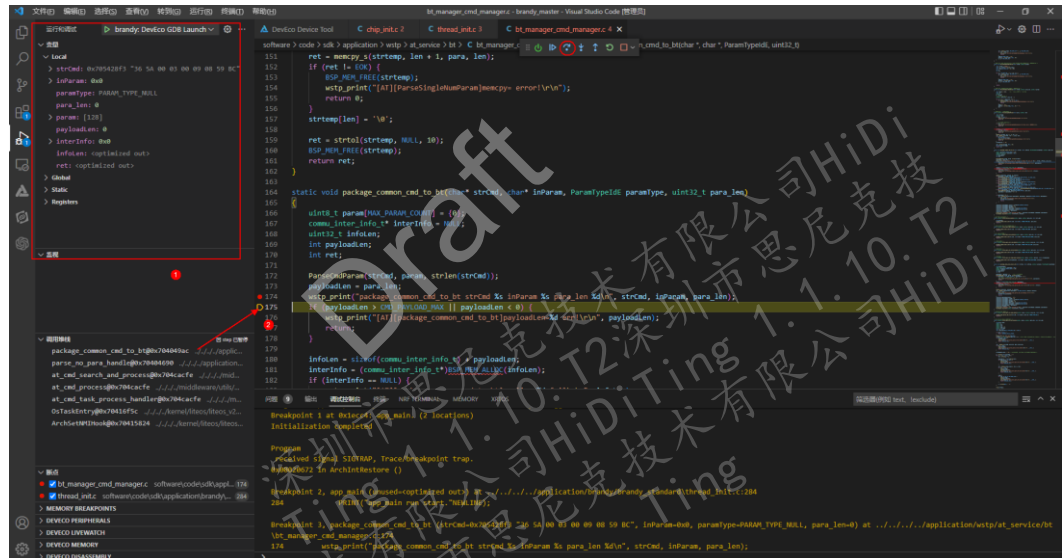
步骤 2 参考 “9.3 启动调试” 章节启动调试后，在串口工具中输入 “AT+BTPSCANOPEN” 命令。



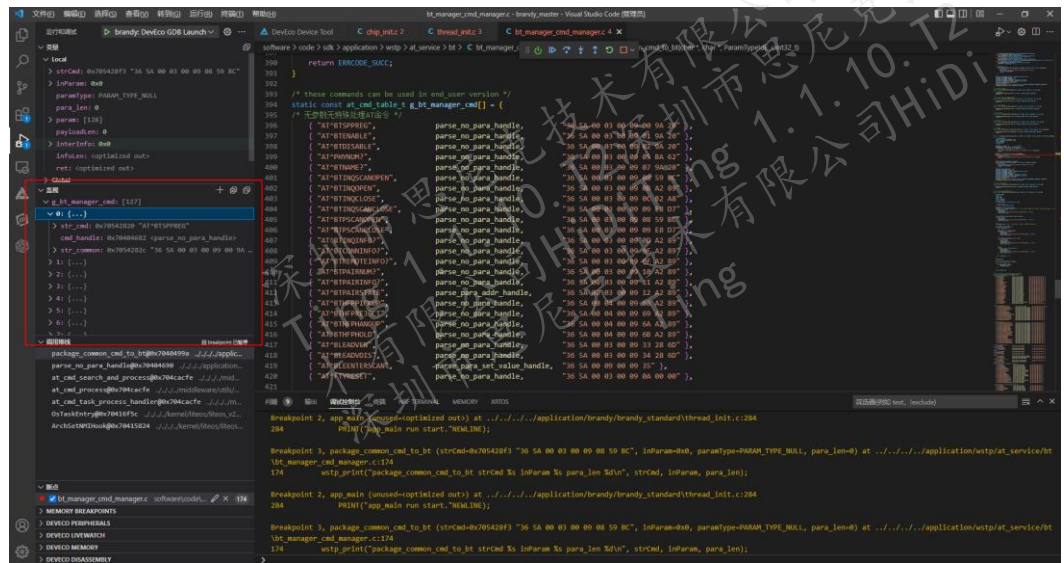
步骤 3 调试器在收到 gdb 信息，断点停留在下图所示断点处。



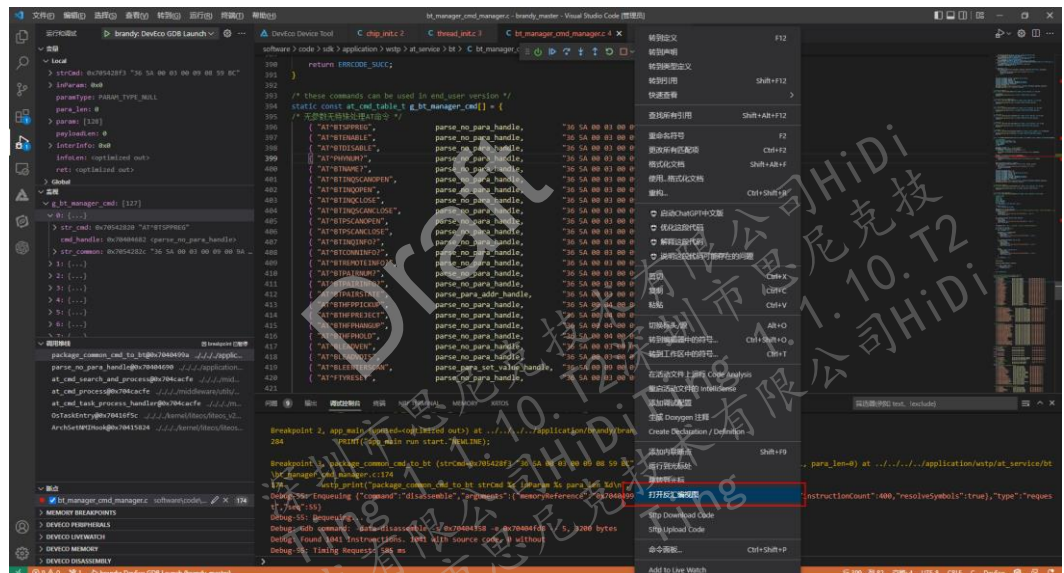
步骤 4 可以使用单步跳过 (F10) 进行单步执行，如下图所示，并查看变量值。



步骤 5 可以使用监视器查看本地变量，或者全局变量值。点击“监视”框图的右上角“+”号可新增监视变量名称，出现对应变量的值。



步骤 6 查看源文件对应的反汇编文件，通过右键点击“打开反汇编视图”。



----结束

10 常见错误

10.1 SDK 根目录路径过长

10.2 路径失效

10.3 调试时出现读取内存地址失败

10.4 IDE 兼容性问题

10.5 替换编译工具链后编译报错

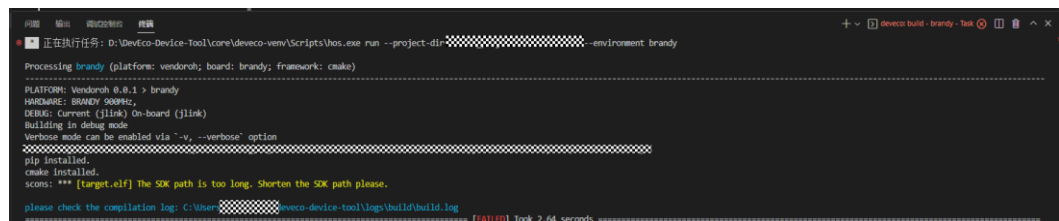
10.6 调试异常打印【warning】Couldn't determine a path for the index cache directory

10.7 自定义编译命令后调试出现问题

10.1 SDK 根目录路径过长

SDK 根目录路径过长：

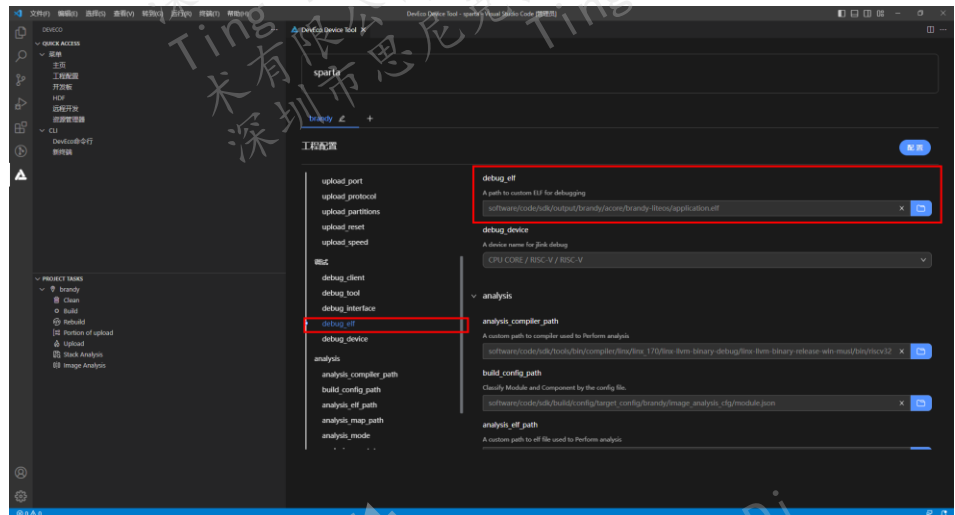
Windows 下路径有 260 的长度限制，过长的路径会导致编译时相关文件无法找到，建议 SDK 代码放到盘符的根目录。当出现如下报错时，请缩短 SDK 存放路径。



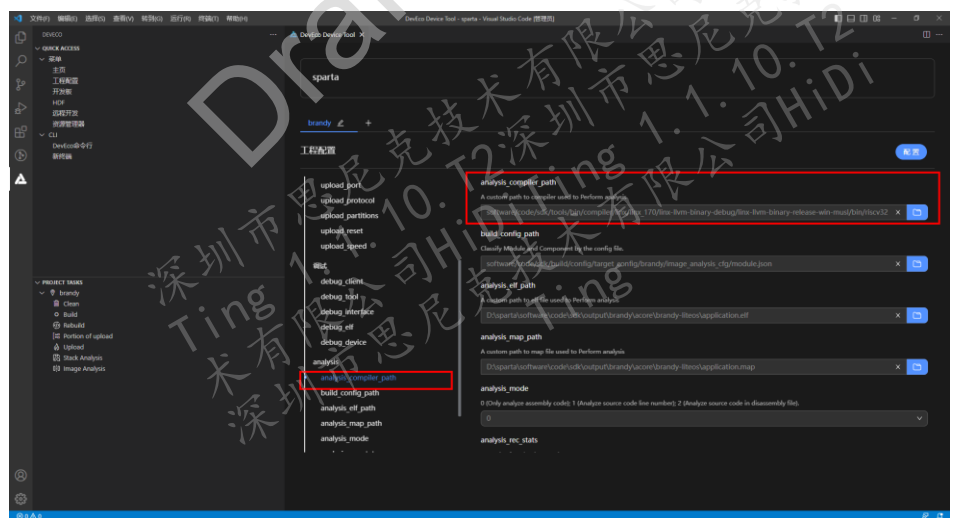
```
正在执行任务: D:\Devco-Device-Tool\core\devco-ven\Scripts\hos.exe run --project-dir ... --environment brandy
Processing brandy (platform: vendoroh; board: brandy; framework: cmake)
PLATFORM: Vendoroh 0.0.1 > brandy
HARDWARE: BRANDY S00P2,
DEBUG: Current (jlink) On-board (jlink)
Building in debug mode
Verbose mode can be enabled via '-v' or '--verbose' option
=====
pip installed.
cmake installed.
scons: *** [target.elf] The SDK path is too long. Shorten the SDK path please.
please check the compilation log: C:\User\... \devco-device-tool\logs\build\build.log
===== [FAIL] Took 2.64 seconds
```


10.2 路径失效

- 问题：导入工程路径问题导致的调试、栈分析、镜像分析等默认路径失效。
- 解决方法：
 - 调试：修改默认的“debug_elf”路径。

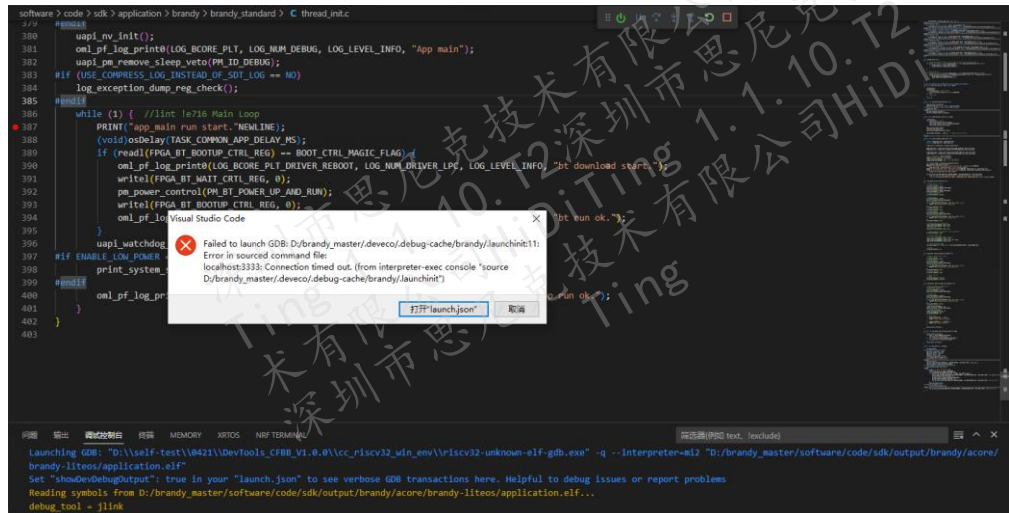


- 栈分析、镜像分析：需要修改编译工具链地址。

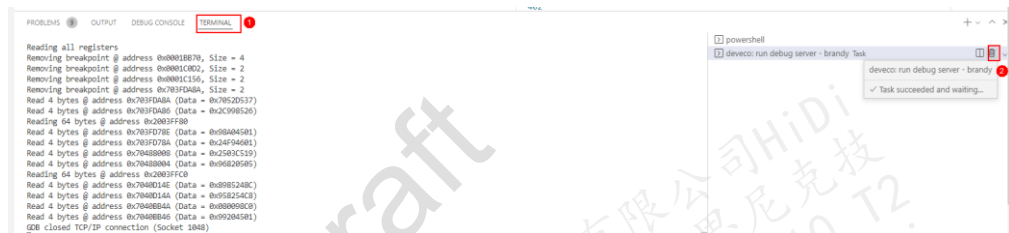


10.3 调试时出现读取内存地址失败

- 问题：单步调试中，出现读取内存地址失败。



- 原因：已经存在一个调试任务没有删除，需要手动关闭该调试任务。
- 解决方法：点击“终端”（英文版“TERMINAL”），在最右侧找到调试任务 run debug server，点击删除按钮，如图所示垃圾桶图标。



10.4 IDE 兼容性问题

- 问题：首次安装 IDE 后一段时间，会提示如下警告“尝试在目标目录创建文件时发生一个错误：拒绝访问。”

错误



选项

C:\Users\ppubconnectpdt\AppData\Local\Pro...\Microsoft
VS Code\unins000.exe

尝试在目标目录创建文件时发生一个错误:
拒绝访问。.

→ 重试(I)

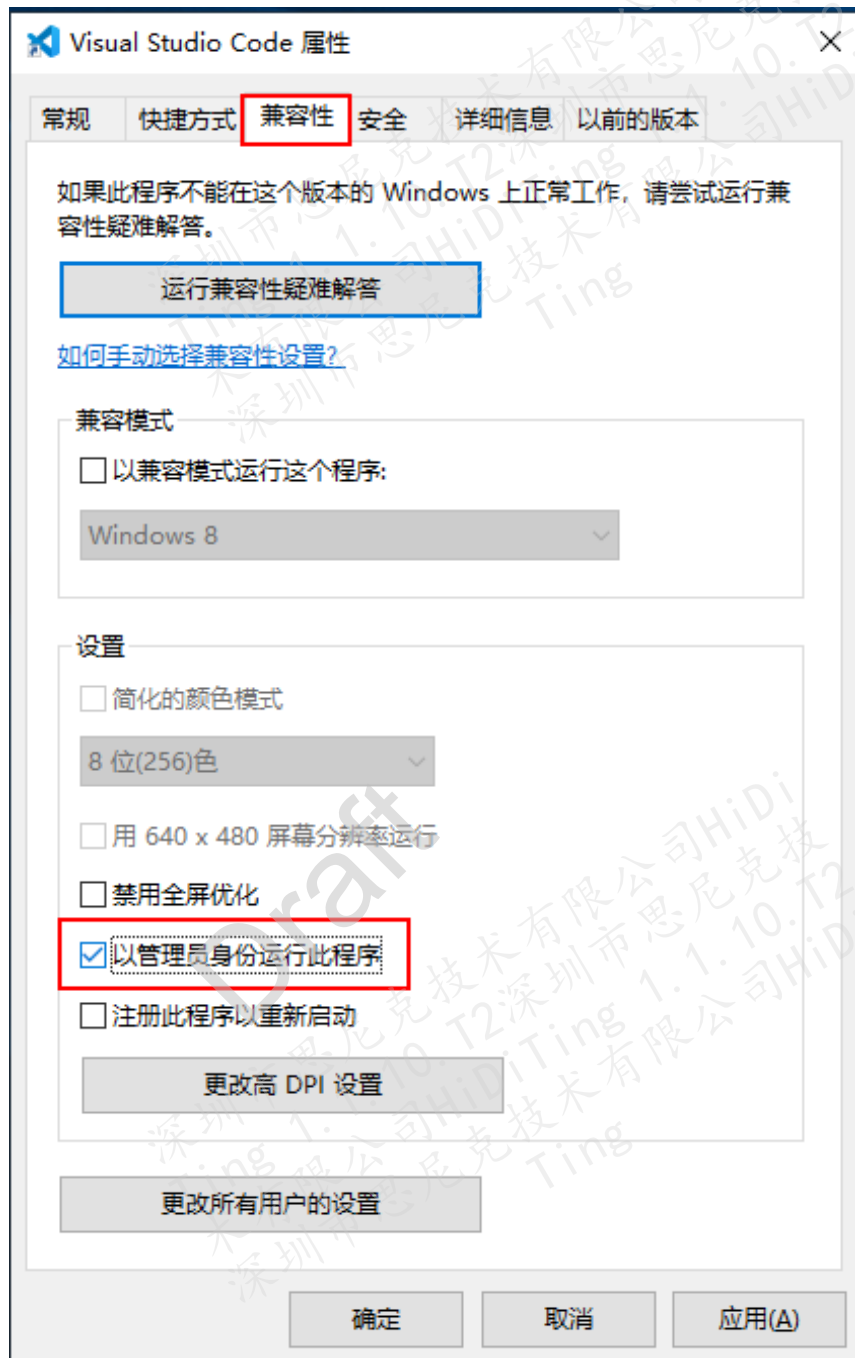
→ 跳过这个文件 (不推荐)(S)

→ 关闭安装程序

- 原因：兼容性问题。
- 解决方案：
 - a. 关闭 VSCode 软件，右键点击 VScode 图标，选择属性。



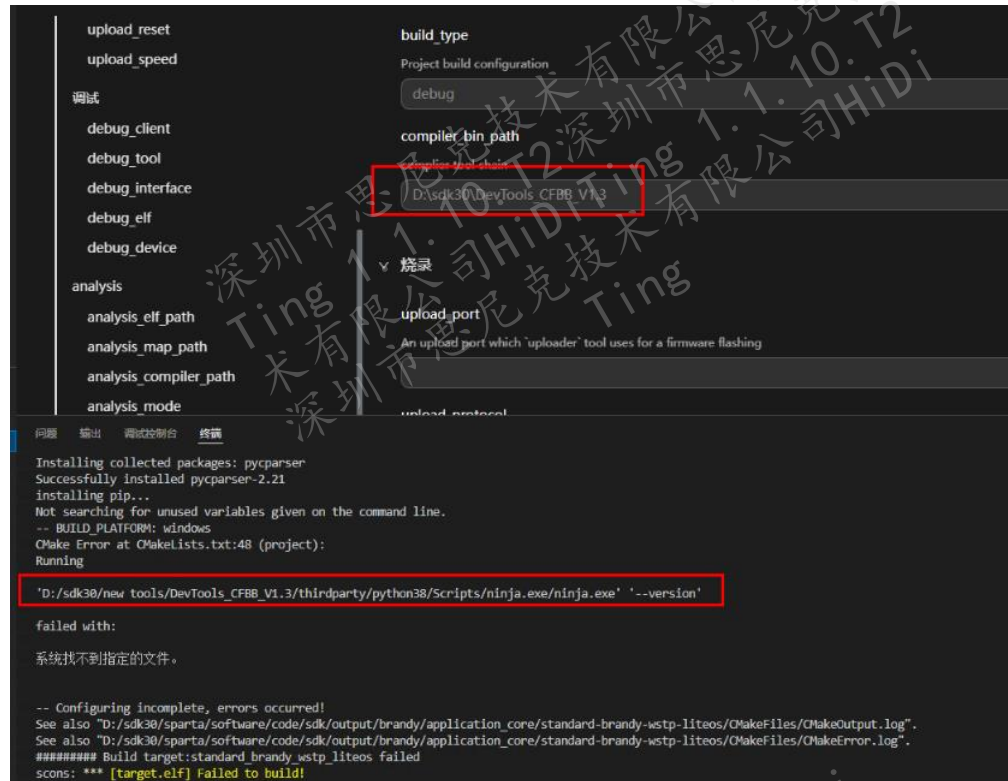
b. 选择“兼容性”标签页，勾选“以管理员身份运行此程序”。



c. 点击“应用”，点击“确定”，双击图标，重新启动 VSCode。

10.5 替换编译工具链后编译报错

- 问题：替换编译工具链后再次点击 Build 进行编译会报错，打印如下。



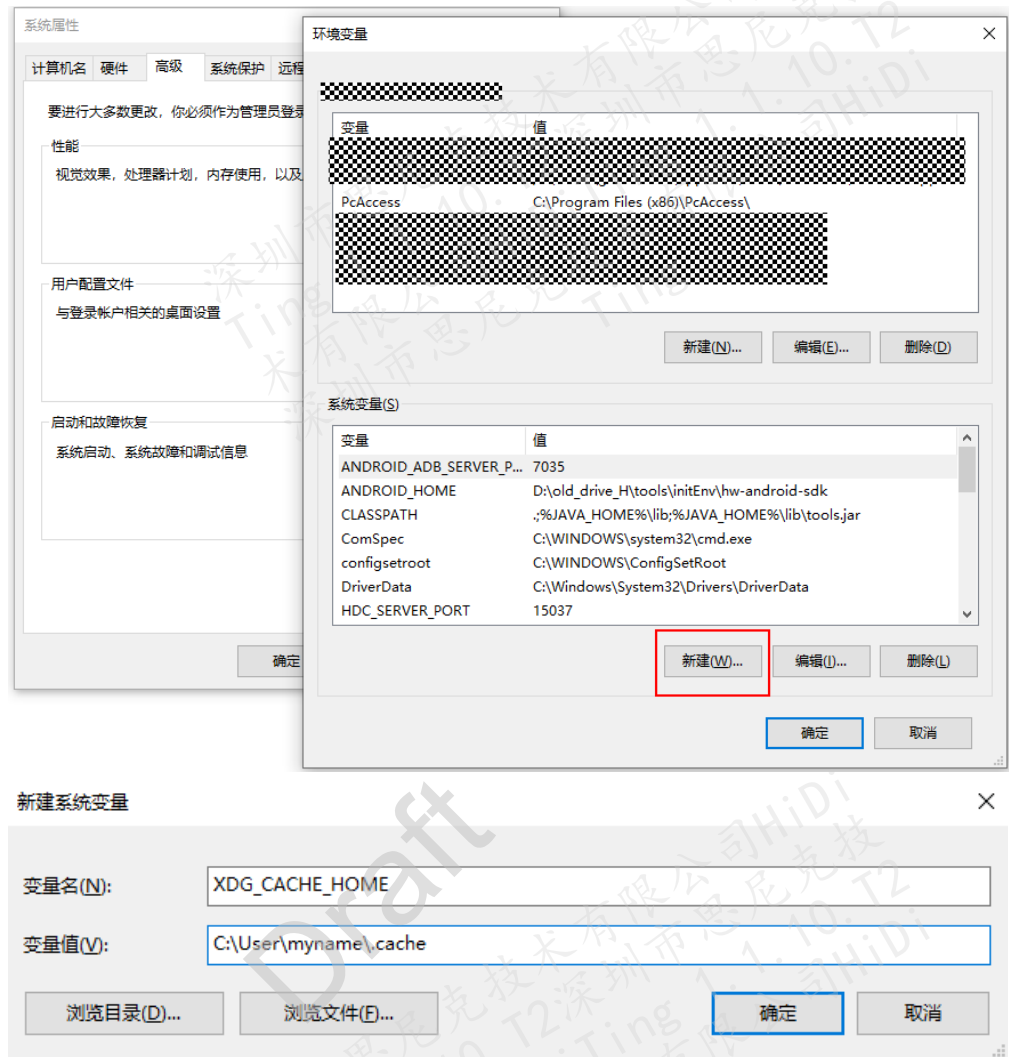
- 原因：在更换工具链之后，原工具包路径失效，导致增量编译需要的工具路径不存在。
- 解决方法：点击 Rebuild 进行全量编译，采用新的工具包路径编译。

10.6 调试异常打印【warning】 Couldn't determine a path for the index cache directory

- 问题：启动调试过程中，如果出现如下异常打印：【warning】 Couldn't determine a path for the index cache directory.

```
Launching gdb: "D:/sdk30/new_tools/DevTools_CFB8_V1.0.0/cc_riscv32_win_env/riscv32-unknown-elf-gdb.exe" -q --interpreter=mi2 "D:/sparta/software/code/sdk/output/brandy/
acore/brandy-liteos/application.elf"
Set "showDebugOutput": true in your "launch.json" to see verbose GDB transactions here. Helpful to debug issues or report problems
undefined: D:/sdk30/new_tools/DevTools_CFB8_V1.0.0/cc_riscv32_win_env/riscv32-unknown-elf-gdb.exe: warning: Couldn't determine a path for the index cache directory.
Reading symbols from D:/sparta/software/code/sdk/output/brandy/acore/brandy-liteos/application.elf...
debug_tool = jlink
Initializing remote target...
```

- 原因：Windows 环境下，gdb 工具需要配置缓存文件夹。
- 解决方法：在“编辑系统环境”→“系统变量”→“新建”，新建环境变量 XDG_CACHE_HOME，其中 myname 为用户实际用户名，如果此目录下没有“.cache”，可新建此目录“.cache”。配置环境变量后，重启 VsCode，重新打开工程，该问题即可解决。



10.7 自定义编译命令后调试出现问题

- 问题：默认编译的 target 为 brandy-liteos，若自定义编译 brandy-freertos，在调试时会出现问题，需要选择对应的 elf 文件路径。
- 原因：elf 文件默认路径为 liteos 系统的路径，使用 freertos 系统时需要重新指定 elf 文件路径。
- 解决方法：在工程配置下设置 “debug_elf” 选择对应系统编译生成的 elf 文件路径。设置完成后，重启 VSCode。

